Inference

101 Guide to Inferring an LLM Model

About me

- Machine Learning Engineer at Nebius
- ex-team lead of Inference team at Yandex.SDG/AVRide

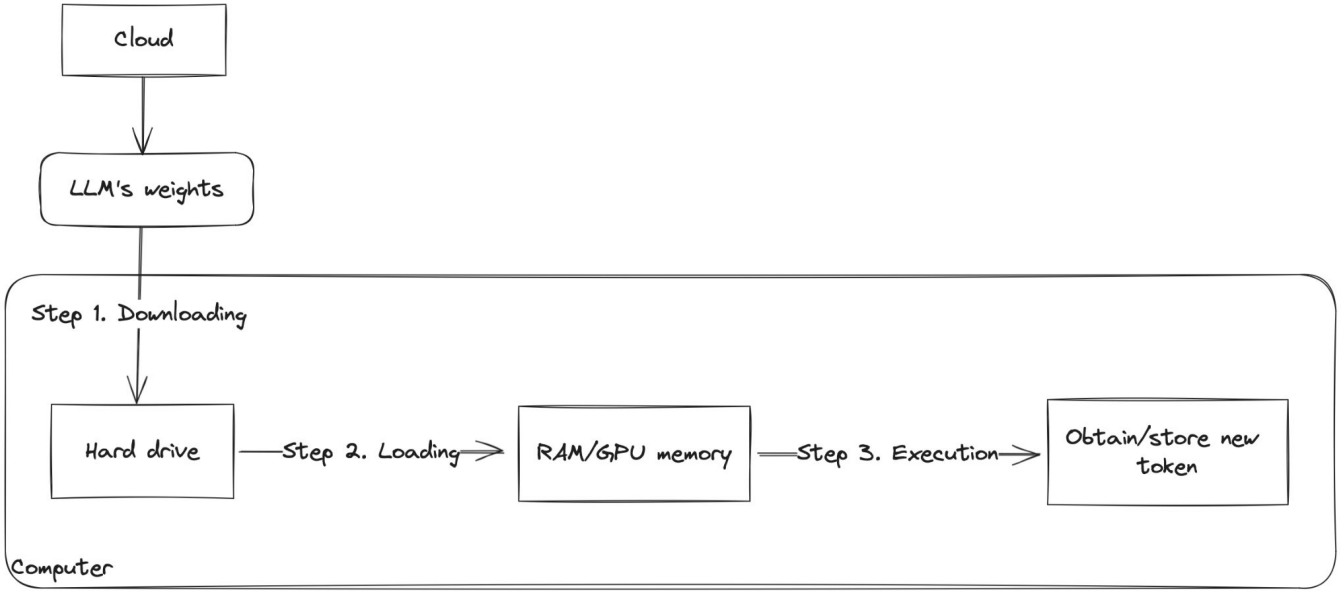
LinkedIn: https://www.linkedin.com/in/sergei-skvortsov

What is LLM inference?

Goal: Generating new tokens for the prompts that the LLM have not seen during the training.

Inference is often optimised for **speed and efficiency**, as it needs to be performed in real-time applications like chatbots or in a batch-like manner, increasing the **utilisation of compute resources**, as seen in large-scale machine learning deployments such as recommendation systems, search engines.

Essentials of LLMs inference.

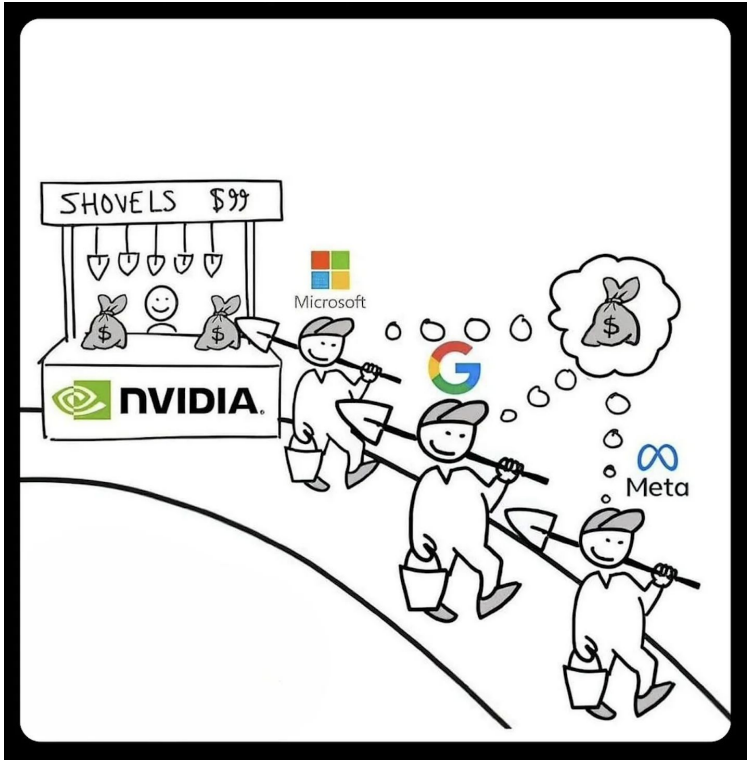


Note: During the inference LLM’s weights are not updated

Why I personally love inference

- We don’t need backward pass anymore!
  Computation of the forward pass much easy and faster
- Easy to measure the result of work
  Every speedup can be easily converted into money
- Room for low-level GPU optimizations

Quiz: What is the main metric of LLM inference?



Main metrics of LLMs inference

Throughput - Number of requests (or tokens) that LLM can process at one second.

Latency - Time from user’s request to response of LLM.

Cost - we want to spend as low as possible.

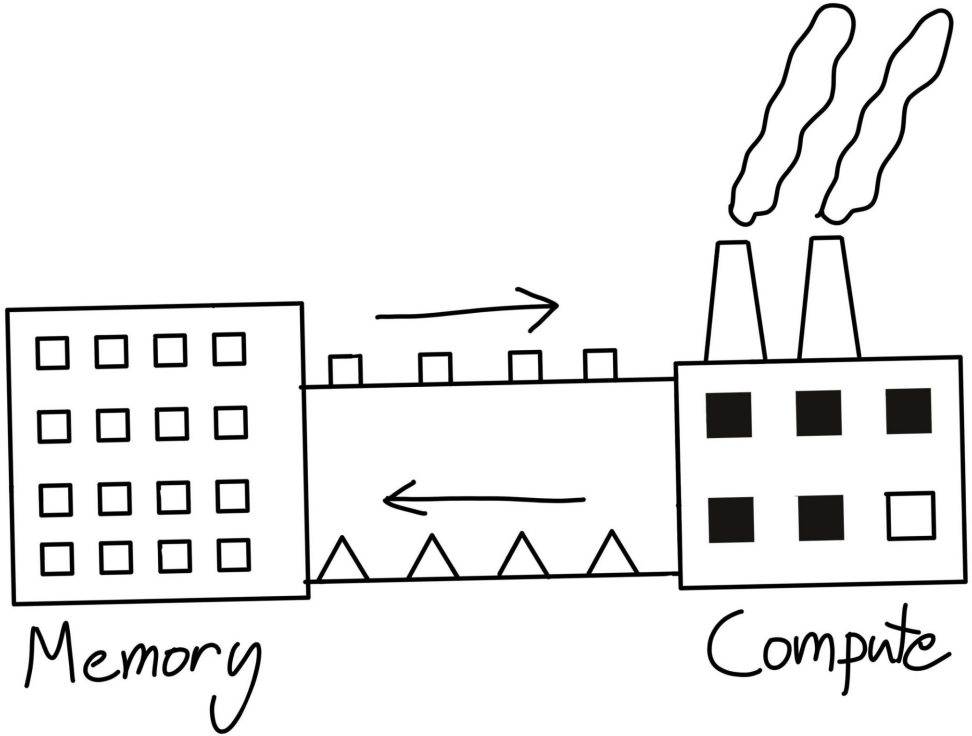
Some advanced metrics of the inference

Time To First Token (TTFT) - How quickly users start seeing the model's output after entering their query.

Time Per Output Token (TPOT) - Time to generate an output token for each user that is querying our system.

Credits: https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices

How GPU works (Steel mill)



Memory == Warehouse

Compute == Factory

Input data == Material (iron)

Weights == Coal

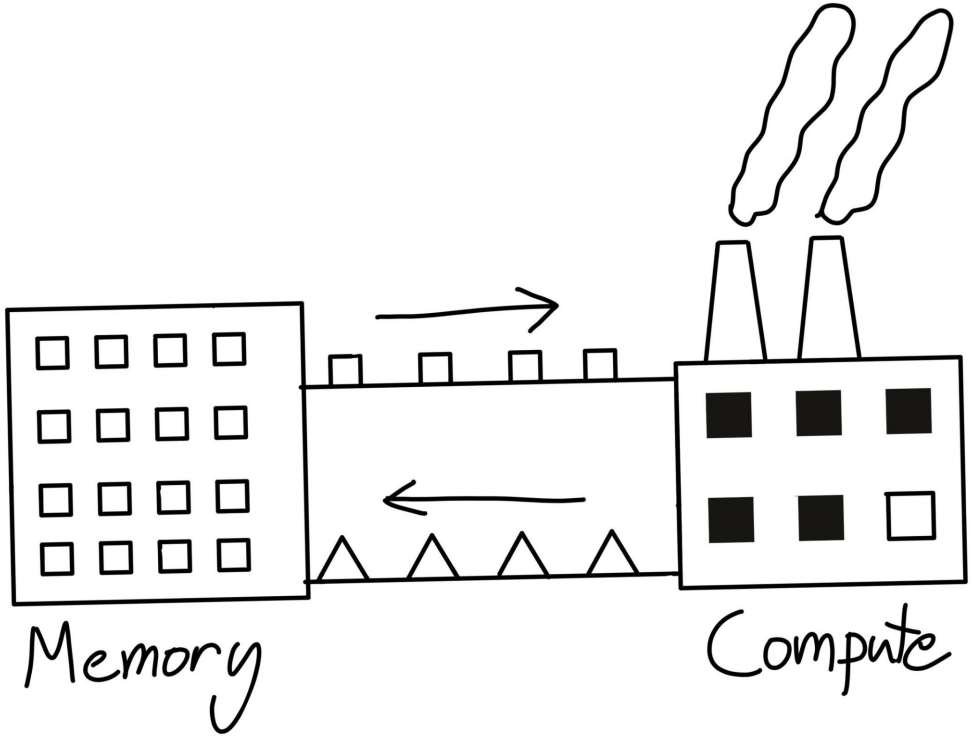
Output data == Product (Steel slab)

Credits: https://horace.io/brrr_intro.html

Steel slab


NB6062-21
NB6061-25
NB6145-21
NC6019-24
NC6028-25
NC6021-61
NC6023-25
NC6024-63
NC6020-25

How GPU works (Steel mill)



Credits: https://horace.io/brrr_intro.html

Memory bandwidth - speed at which these materials are delivered from the warehouse to the factory

Peak FLOPS - speed of processing the raw material into the products

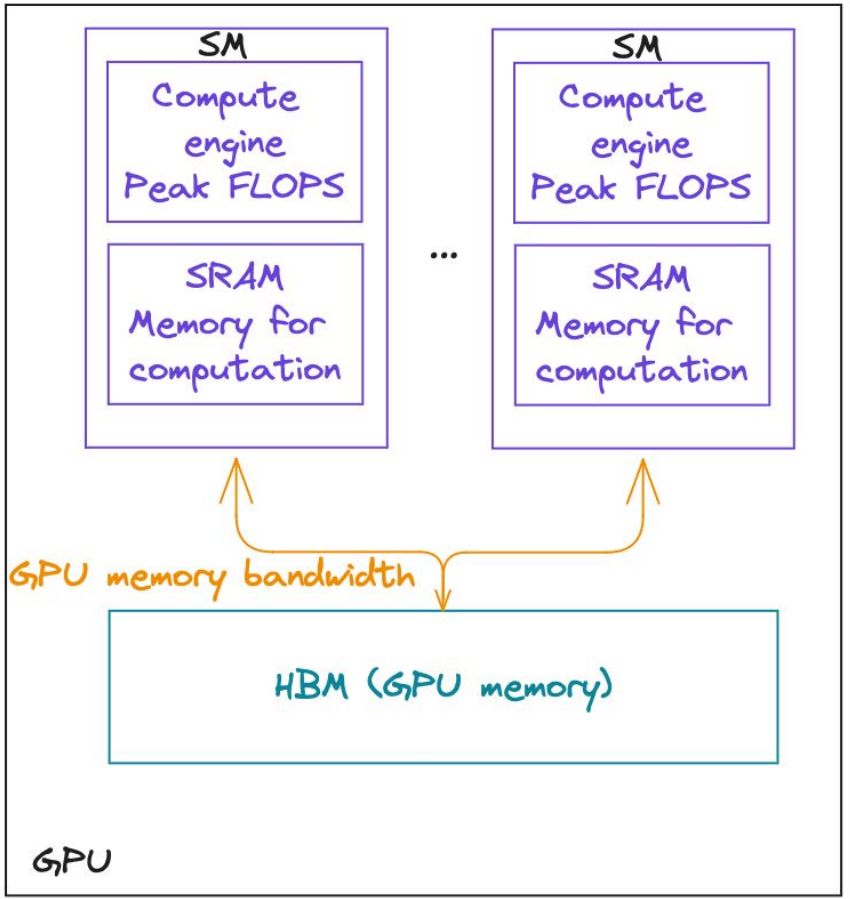
How GPU works

Compute engines == blast furnace

SRAM == local storage

Memory bus == conveyor belt

HBM == warehouse



Important parameters of GPUs

Peak FLOPS

GPU memory size

GPU memory bandwidth

NVIDIA A100 TENSOR CORE GPU SPECIFICATIONS (SXM4 AND PCIE FORM FACTORS)

	A100 40GB PCIe	A100 80GB PCIe	A100 40GB SXM	A100 80GB SXM
FP64	9.7 TFLOPS			
FP64 Tensor Core	19.5 TFLOPS			
FP32	19.5 TFLOPS			
Tensor Float 32 (TF32)	156 TFLOPS | 312 TFLOPS*			
BFLOAT16 Tensor Core	312 TFLOPS | 624 TFLOPS*			
FP16 Tensor Core	312 TFLOPS | 624 TFLOPS*			
INT8 Tensor Core	624 TOPS | 1248 TOPS*			
GPU Memory	40GB HBM2	80GB HBM2e	40GB HBM2	80GB HBM2e
GPU Memory Bandwidth	1,555GB/s	1,935GB/s	1,555GB/s	2,039GB/s

Example: https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf

Bottlenecks of computing

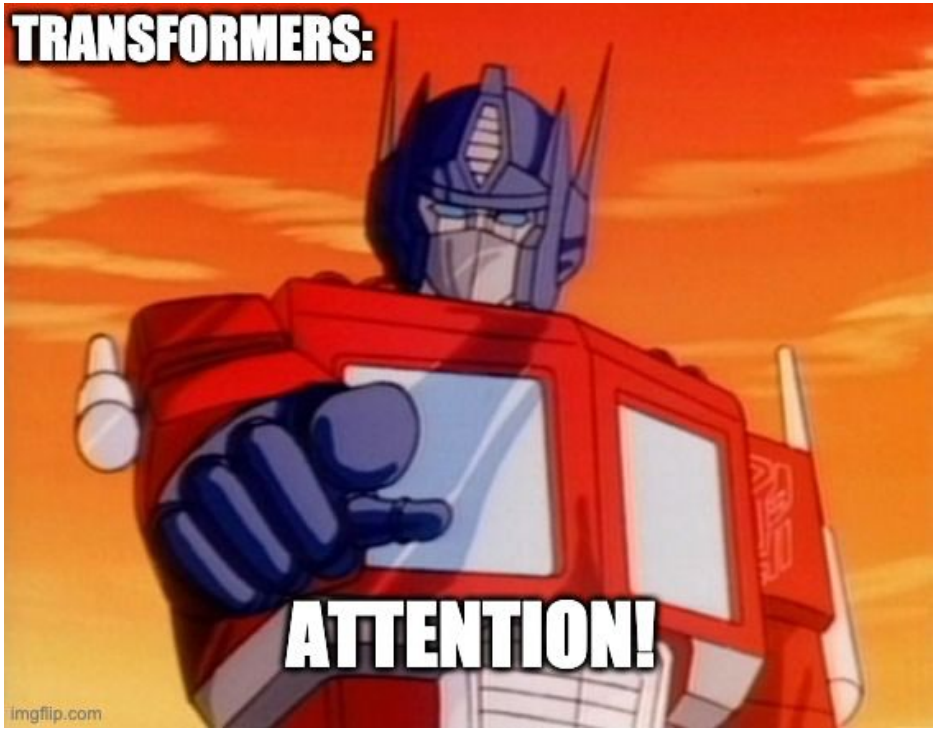
Compute bound program:

time of computing >> time of data movement

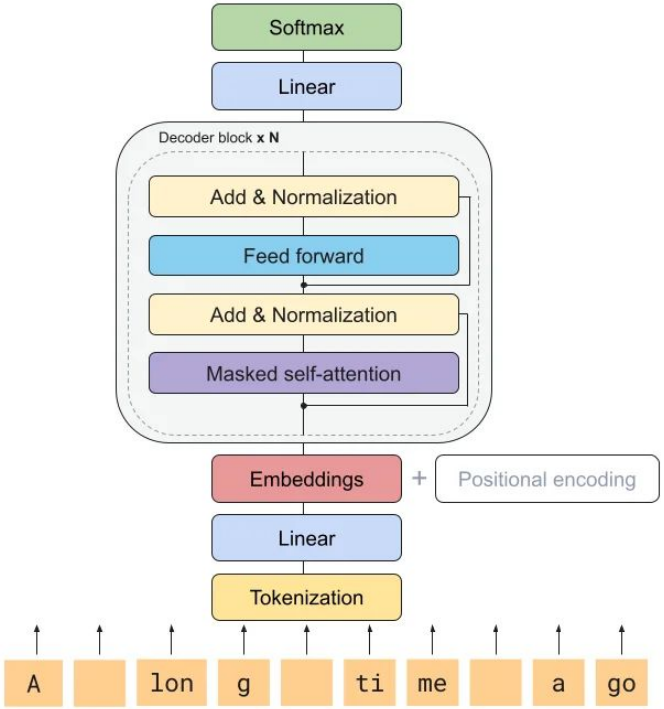
Memory bound program:

time of computing << time of data movement

Quiz: Layers of decoder-only transformers



Decoder-only transformer architecture



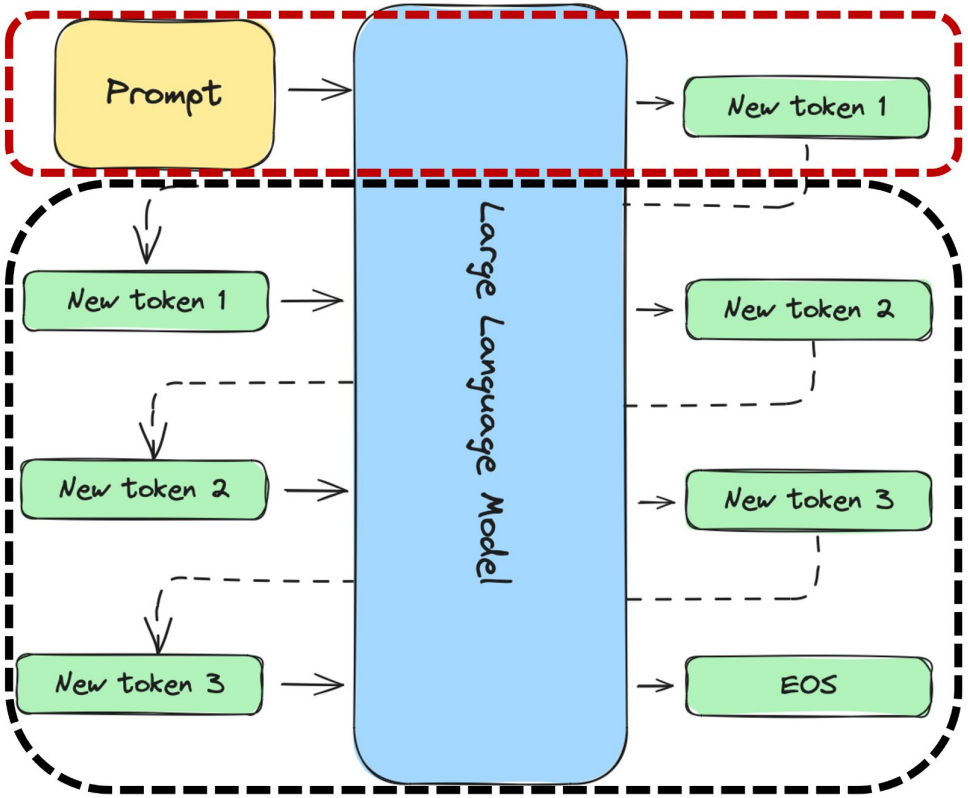
How LLM inference works

Two main stages:

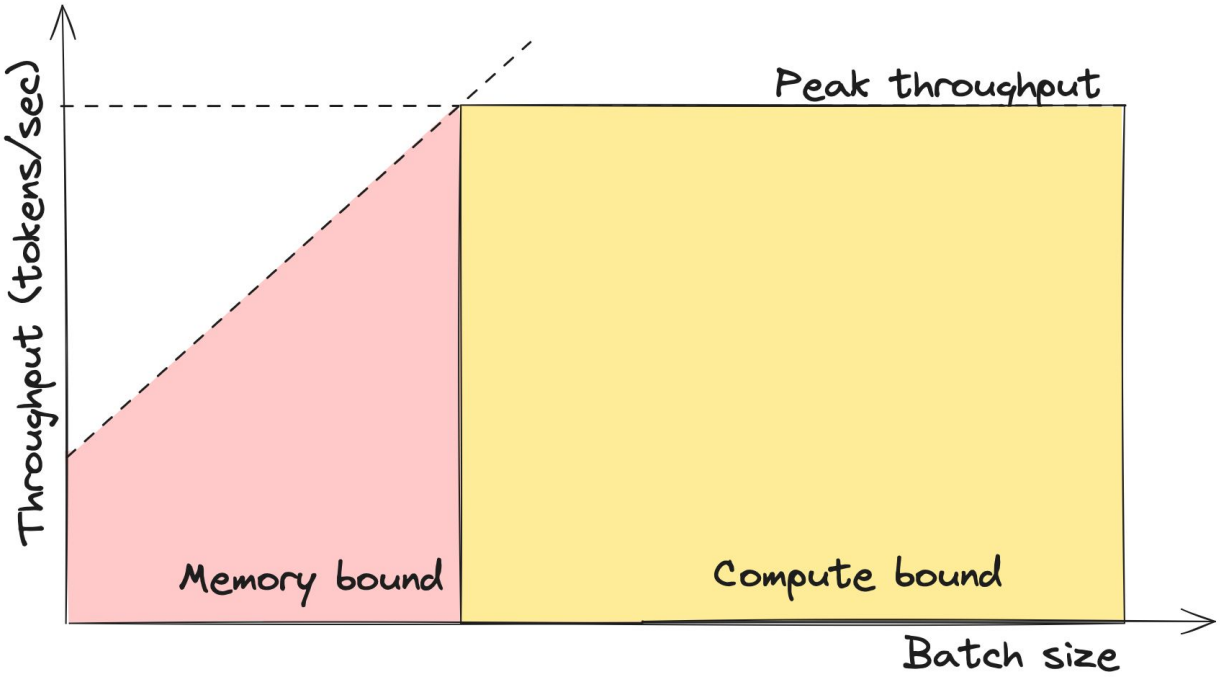
- **Cache prefill** (Prompt processing)
- **Autoregressive decoding**

Step 1: Prompt Processing

Step 2: Autoregressive Decoding



Compute/memory bound of decoding stage



What’s LLMs memory consist of?

The main parts of LLM’s memory consumption:

- **Weights of LLM**
  - Depends on precision, usually bf16 or fp16
- **KV-Cache**
  - Depends on batch size, context lengths and quantization.
- **Activation**
  - Memory, used to store intermediate outputs.
  - Depends on framework implementation and precision
  - Usually we assume it as constant and don’t consider in calculation

How to calculate memory utilisation of LLM?

For a simple approximation, we can consider that generating one token requires loading all our weights through the memory bus. It means, that data movement M can be estimated by the formula:

$$M = N * B,$$

where:
N - number of parameters of our LLM,
B - number of bytes used to store one parameter.

Warning: For simplicity, we are assuming the context length is very short, so we can ignore it.

How to calculate number float point operations of LLM?

For very rough approximation we will use formula:

$$\text{FLOPs} = 2 * N * \text{batch\_size},$$

where N is a number of parameters of our LLM.

Intuition of the formula: We need to multiply and then sum result through all the parameters

Credits: https://kipp.ly/transformer-inference-arithmetic/#flops-counting

Rough estimation of the inference time

Formulas for calculating compute and memory times:

$$\text{time\_of\_computation} = \frac{\text{FLOPs}}{\text{GPU\_peak\_FLOPs}}$$

$$\text{time\_of\_data\_movement} = \frac{M}{\text{GPU\_memory\_bandwidth}}$$

Based on these formulas, we can calculate estimation throughput:

$$\text{throughput} = \frac{\text{batch\_size}}{\text{time\_of\_computation} + \text{time\_of\_data\_movement}}$$

Rough estimation of the inference time

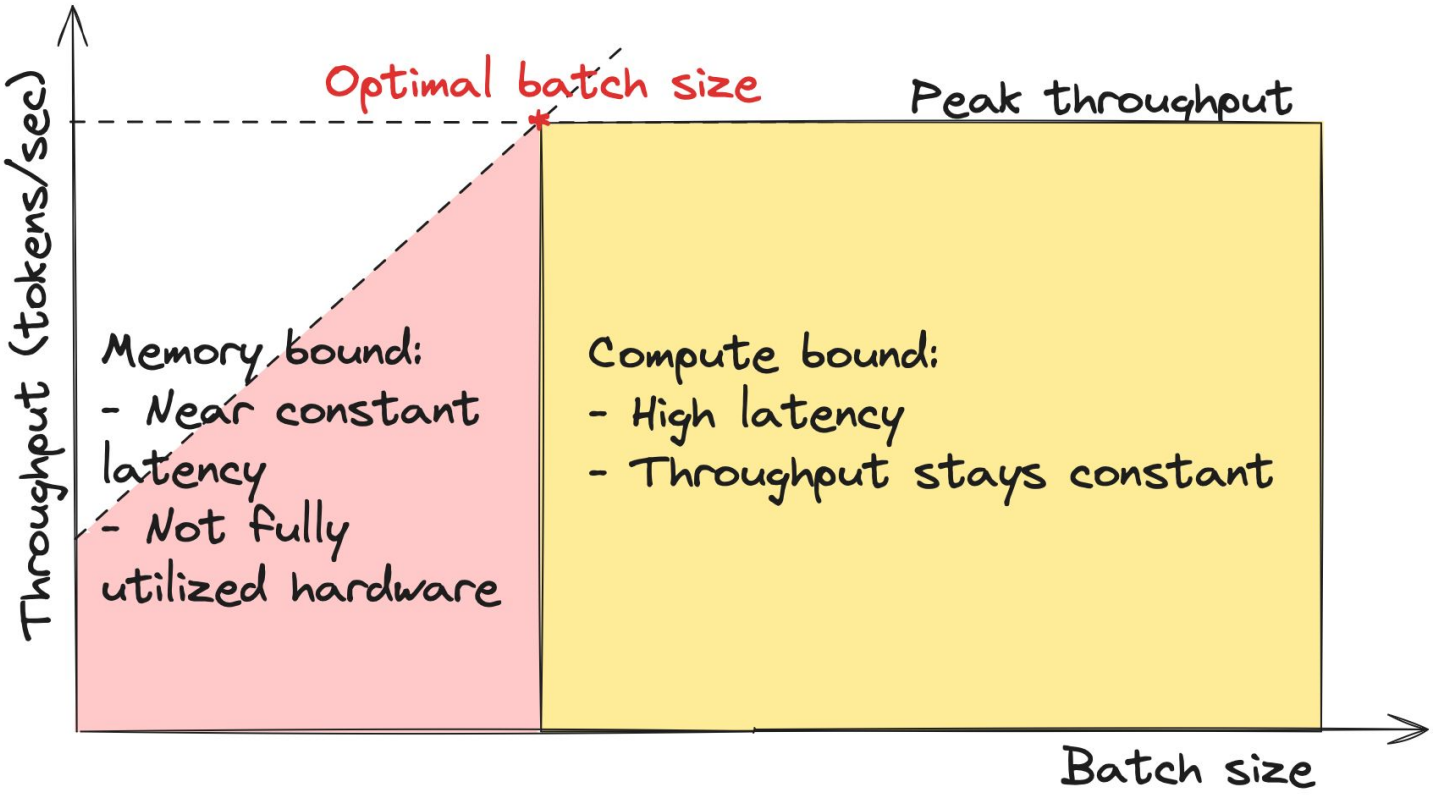
We also can understand do we memory or compute bound:

Memory bound: time_of_computation << time_of_data_movement

Compute bound: time_of_computation >> time_of_data_movement

But why do we need this?

Optimal value of batch_size



Finding an optimal batch_size value

Optimal batch size point is:

time_of_computation == time_of_data_movement

and can be calculated as:

$$\text{optimal\_batch\_size} = \frac{\text{GPU\_peak\_FLOPs} \times B}{2 \times \text{GPU\_memory\_bandwidth}}$$

For A100 GPU in case of float16 optimal_batch_size is:

$$\text{optimal\_batch\_size} = \frac{312\,\text{TFLOPS}}{1.935\,\text{TB/s}} = 160$$

LLM inference is memory bound

As we said before, LLM memory consumption consist of:

- Memory for weights
- Memory for cache (which depends on batch size and length of context)

The real batch size that can be used for llama 8b is much lower than 160.

It means that inference of LLMs is **Memory bound**.

Pros and cons of this method

- Oversimplified
  - We skip the FLOPs of attention operations and the normalization layer.
  - LLM architecture is also not taken into account during calculation FLOPs of MLP.
  - We calculated the memory movement of parameters only (and ignored the KV cache).
- Number of LLM parameters is not included in the final formula
- We did not take memory limitations into account.

- Simple
  - Great to understand how the LLM inference works

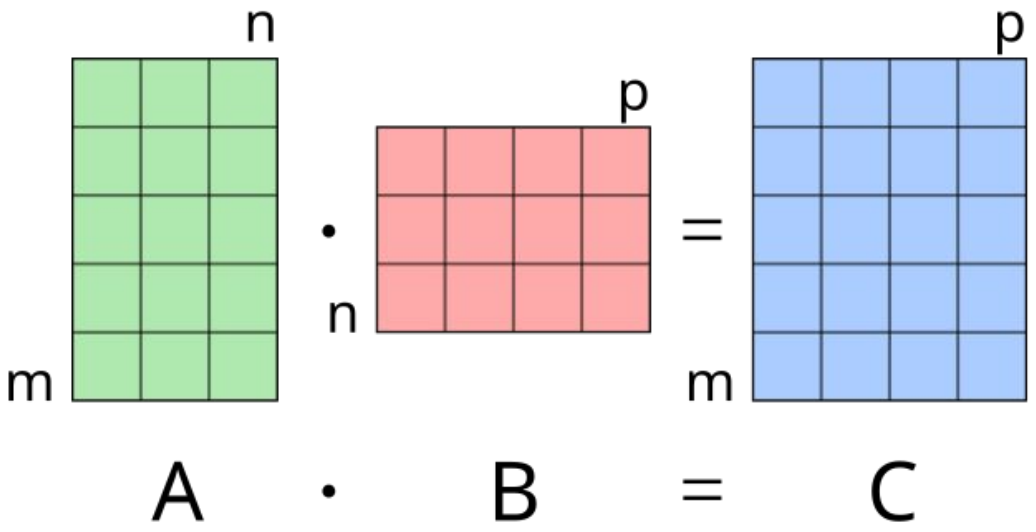
Advanced LLM math

For every element of the matrix C we need to make n multiplications and n - 1 additions =>

n + n - 1 = 2n - 1

The number of FLOPs in (m, n) x (n, p) matrix multiplication operation:

FLOPs = m * p * (2n -1) => 2mnp



FLOPs in attention layer

Query projection: (seq_len, hid_dim) x (hid_dim, num_heads * head_dim) =>

$$C_{\text{query}} = 2 \times \text{seq\_len} \times \text{hid\_dim} \times \text{num\_heads} \times \text{head\_dim}$$

Key/Value projection: (seq_len, hid_dim) x (hid_dim, num_kv_heads * head_dim) =>

$$C_{\text{kv}} = 2 \times \text{seq\_len} \times \text{hid\_dim} \times \text{num\_kv\_heads} \times \text{head\_dim}$$

Output projection: (seq_len, num_heads * head_dim) x (num_heads * head_dim, hid_dim) => $C_{\text{out}} = 2 \times \text{seq\_len} \times \text{hid\_dim} \times \text{num\_heads} \times \text{head\_dim}$

Total FLOPs:

$$C_{\text{attn\_proj}} = 4 \times \text{hid\_dim} \times \text{seq\_len} \times \text{head\_dim} \times (\text{num\_heads} + \text{num\_kv\_heads})$$

FLOPs in attention operation

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N\times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N\times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N\times d},$$

FLOPs in S multiplication: (seq_len, head_dim) x (head_dim, seq_len) =>

$$C_{\mathrm{S}} = 2 \times \mathrm{seq\_len}^2 \times \mathrm{head\_dim}$$

FLOPs in O multiplication: (seq_len, seq_len) x (seq_len, head_dim) =>

$$C_{\mathrm{O}} = 2 \times \mathrm{seq\_len}^2 \times \mathrm{head\_dim}$$

FLOPs in an attention operation:

$$C_{\mathrm{attn\_op}} = \mathrm{num\_head} \times 4 \times \mathrm{seq\_len}^2 \times \mathrm{head\_dim}$$

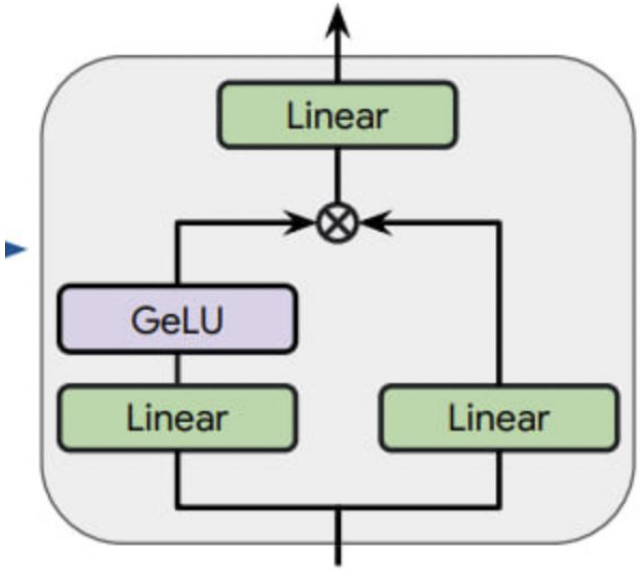
FLOPs in MLP layer

We have 3 multiplications:

1: (seq_len, hid_size) x (hid_size, intermediate_size)

2: (seq_len, hid_size) x (hid_size, intermediate_size)

3: (seq_len, intermediate_size) x (intermediate_size, hid_size)



(b) Gated MLP block

$$C_{\text{MLP}} = 6 \times \text{seq\_len} \times \text{hid\_size} \times \text{intermediate\_size}$$

FLOPs in Embedding/Unembedding layers

FLOPs in embedding layer: 0 (lookup table)

FLOPs in unembedding layer: $(\text{seq\_len}, \text{hid\_dim}) \times (\text{hid\_dim}, \text{vocab\_size}) \Rightarrow$

$$C_{\text{unemb}} = 2 \times \text{seq\_len} \times \text{hid\_dim} \times \text{vocab\_size}$$

FLOPs in one inference pass

$$\text{FLOPs} = \text{batch\_size} \times (\text{num\_layers} \times (C_{\text{attn}} + C_{\text{mlp}}) + C_{\text{unemb}})$$

$$M = M_{\text{param}} + M_{\text{kv\_cache}} \times \text{batch\_size}$$

Llama 3.1 8B FLOPs calculation

For Llama3.1 8b model:

hidden_size: 4096
intermediate_size: 14336
num_heads: 32
num_kv_heads: 8

head_size: hidden_size // num_heads = 128
seq_len: 8192 (in our case)
vocab_size: 128256
num_layers: 32

$$C_{\text{attn\_proj}} = 4 \times 4096 \times 1 \times 128 \times (32 + 8) = 83\,886\,080$$

$$C_{\text{attn}} = 32 \times 4 \times 8192 \times 1 \times 128 = 134\,217\,728$$

$$C_{\text{MLP}} = 6 \times 1 \times 4096 \times 14336 = 352\,321\,536$$

$$C_{\text{unemb}} = 2 \times 1 \times 4096 \times 128256 = 1\,050\,673152$$

$$\text{FLOPs} = 19.3 \times 10^9 \times \text{batch\_size}$$

Llama 3.1 8B memory movement calculation

$$M_{\text{param}} = 8 \times 10^9 \times 2 = 16 \times 10^9$$

$$M_{\text{kv\_cache}} = 2 \times 2 \times \text{num\_layers} \times \text{seq\_len} \times \text{num\_kv\_heads} \times \text{head\_size}$$

$$M_{\text{kv\_cache}} = 2 \times 2 \times 32 \times 8192 \times 8 \times 128 = 1 \times 10^9$$

$$M = 16 \times 10^9 + 1 \times 10^9 \times \text{batch\_size}$$

Optimal batch_size value for llama 3.1 8B

Formulas for calculating compute and memory times:

$$\text{time\_of\_computation} = \frac{\text{FLOPs}}{\text{GPU\_peak\_FLOPs}}$$

$$\text{time\_of\_data\_movement} = \frac{M}{\text{GPU\_memory\_bandwidth}}$$

Optimal batch size point is:

time_of_computation == time_of_data_movement

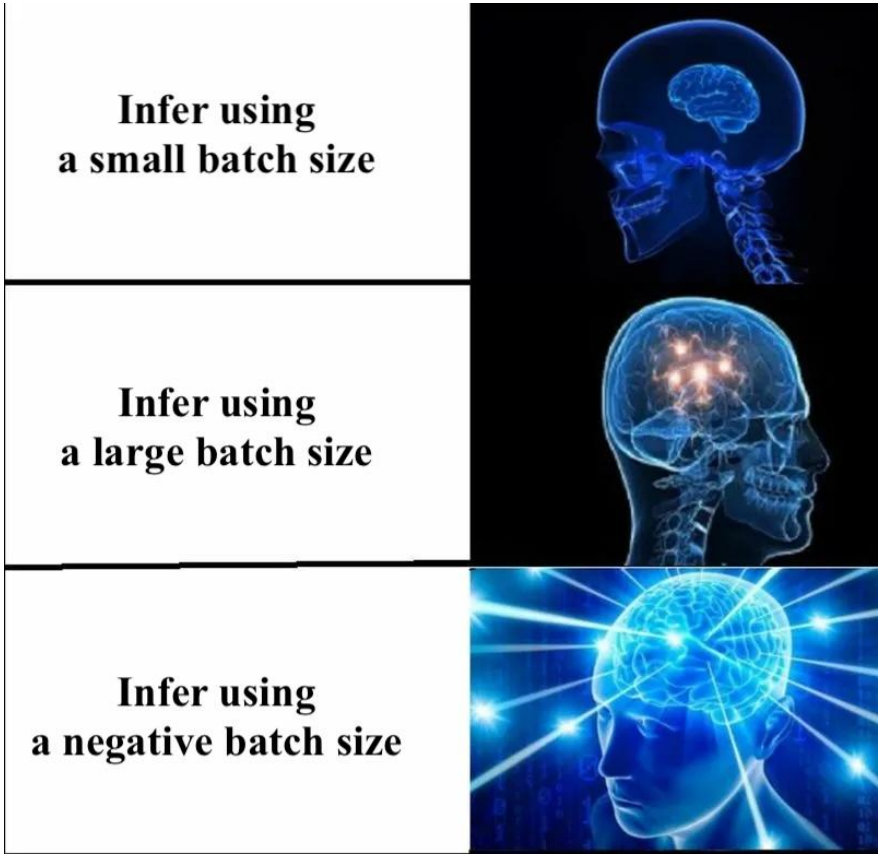
$$\frac{\text{batch\_size} \times \text{FLOPs}}{\text{GPU\_peak\_FLOPs}} = \frac{M_{\text{param}} + \text{batch\_size} \times M_{\text{kv\_cache}}}{\text{GPU\_memory\_bandwidth}}$$

Optimal batch_size value for llama 3.1 8B

Optimal_batch_size = -16.7

What does that mean?

What the problem with our computational model?



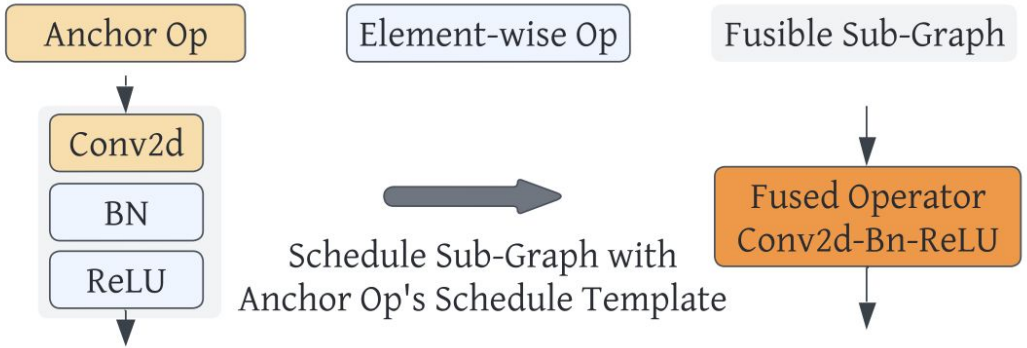
Quiz: How to speedup the inference?

Neural network compilers

The main tasks of NN compilers are to simplify and speed up the compute graph of a NN model, reducing latency, memory requirements, and increasing throughput

It can be achieved by:

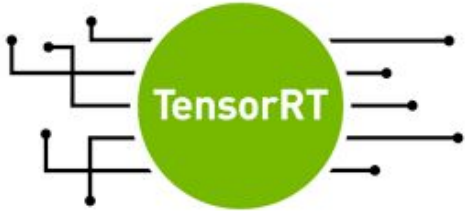
- Fusing operations
- Using more efficient implementations of some operations
- Simplifying the graph by reducing unused parts



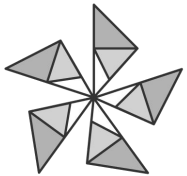
Possible fusing of operations in TVM compiler
Credits: https://arxiv.org/pdf/2210.09603

List of popular neural networks compilers

- TensorRT (TensorRT-LLM)
- vLLM (not a compiler)
- Accelerated Linear Algebra (XLA, OpenXLA)
- OnnxRuntime
- TVM
- …

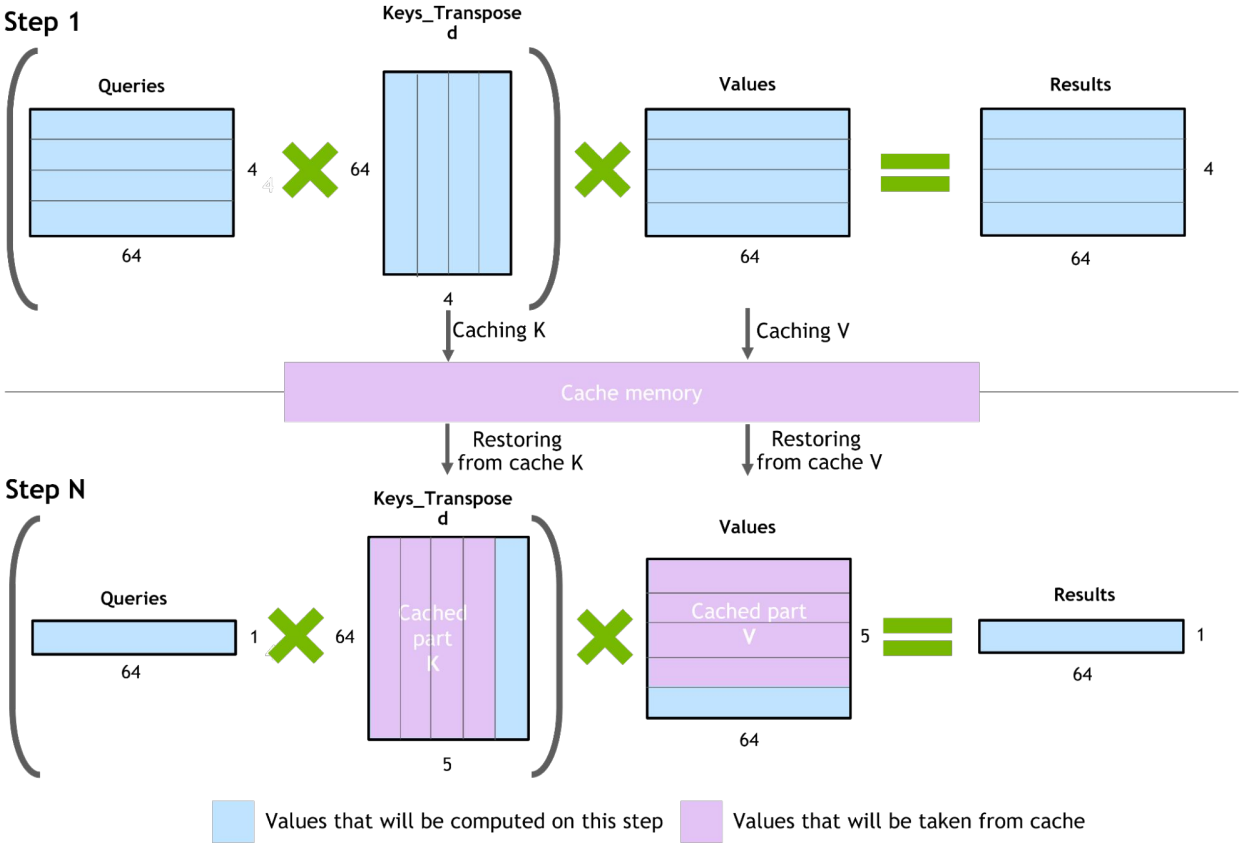


OpenXLA



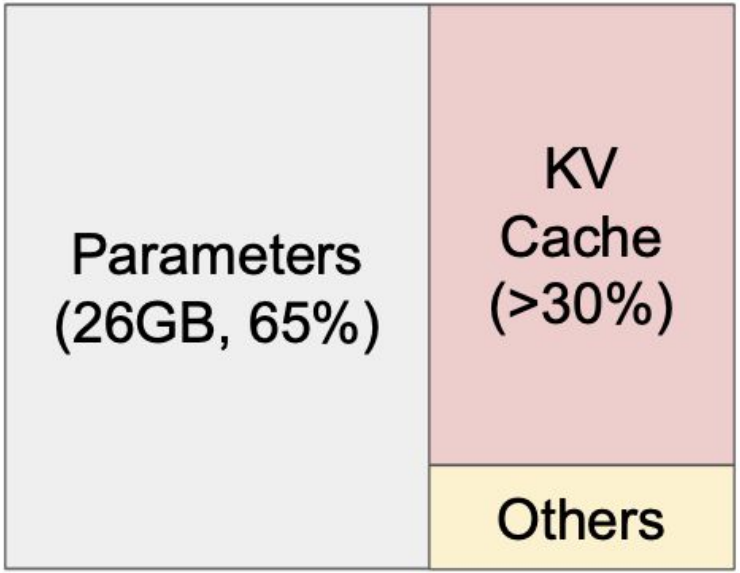
ONNX
RUNTIME

Model optimisations: KV-cache



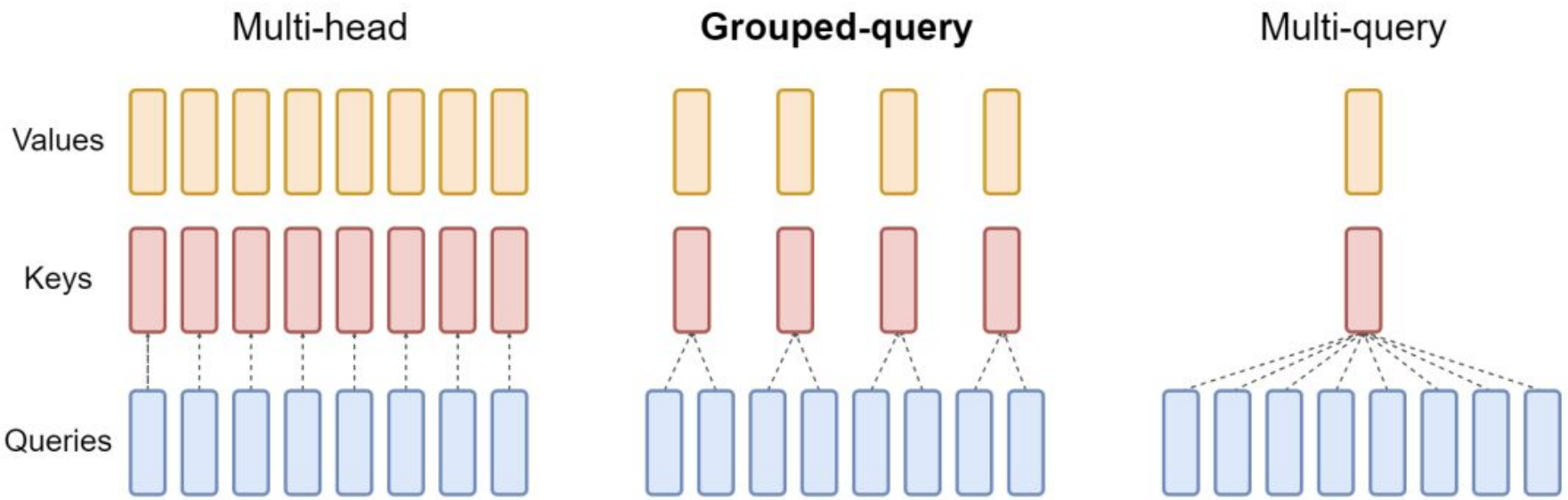
Problems with KV-cache

- Consume a lot of memory. For example, for 13B models it can take about 30% of all GPU memory
- When managed inefficiently, this memory can be significantly wasted by fragmentation and redundant duplication, limiting the batch size.



Memory layout of 13B LLM model.
Credits: https://arxiv.org/pdf/2309.06180

Grouped Query Attention

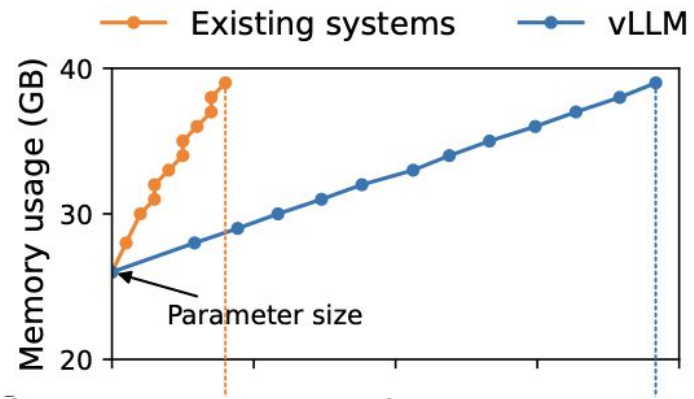


Credits: https://arxiv.org/pdf/2305.13245

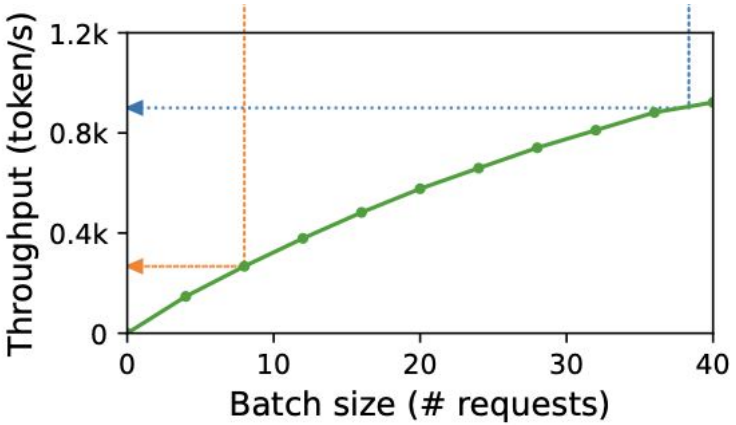
PagedAttention (vLLM)

vLLM - Open source LLM serving system that achieves:

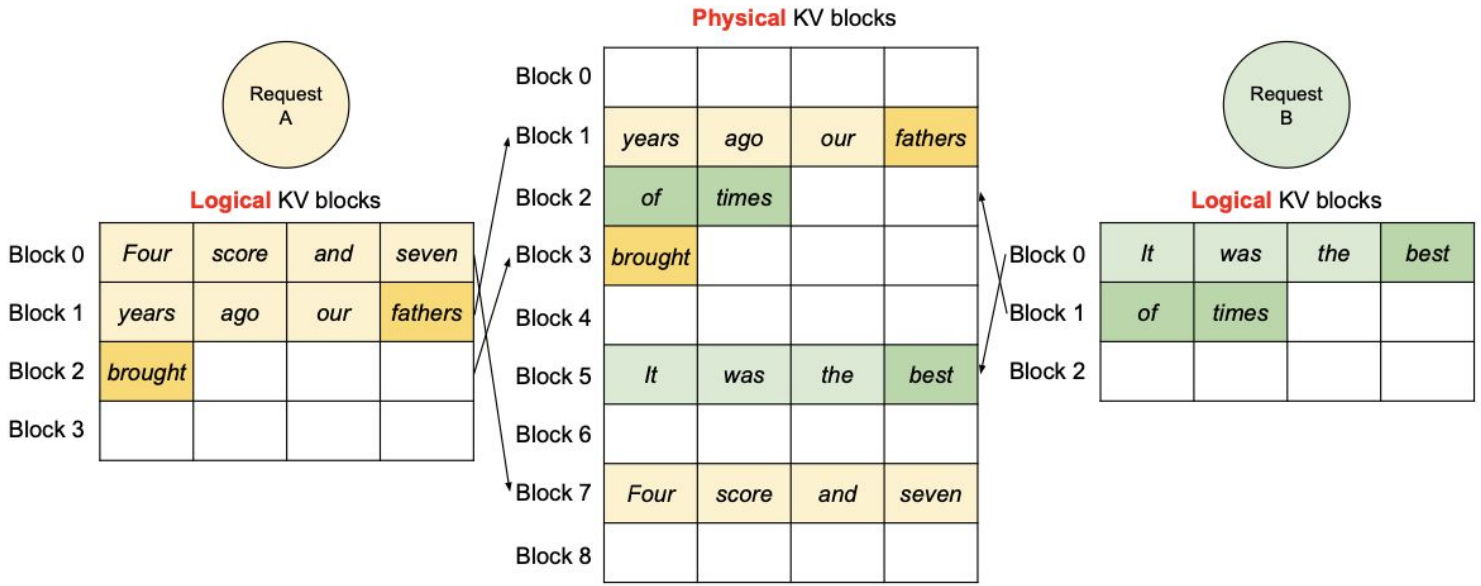
1. Near-zero waste in KV cache memory
2. Flexible sharing of KV cache within and across requests to further reduce memory usage.



Credits: https://arxiv.org/pdf/2309.06180



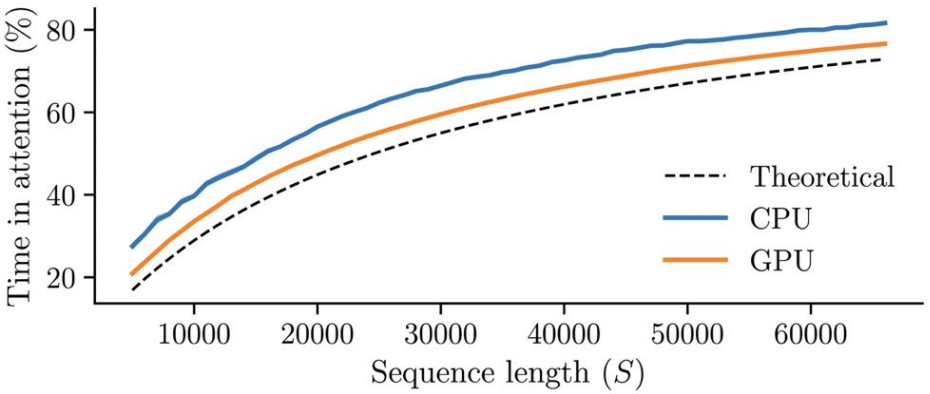
PagedAttention in depth



Credits: https://arxiv.org/pdf/2309.06180

What’s wrong with the attention?

Llama 2 7B, A100 latency	S = 1024	S = 128k
KV cache data transfer	0.3ms (3.8%)	43.0ms (82.9%)
Attention compute	0.0ms (0.0%)	0.2ms (0.4%)
Parameter data transfer	8.6ms (95.7%)	8.6ms (16.6%)
Non-attention compute	0.0ms (0.5%)	0.0ms (0.1%)
Total	8.9ms	51.8ms



Credits: https://graphcore-research.github.io/posts/sparq/

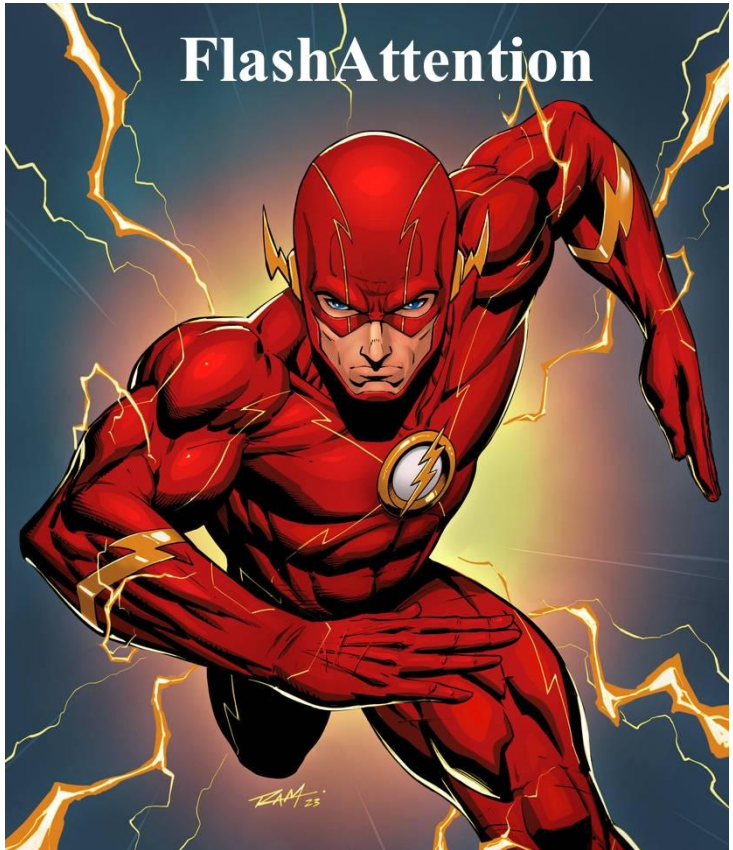
Standard attention problem

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

1. Takes O(N^2) memory and calculation. For example, if N = 32768 (like in Mixtral), matrix S require about 2Gb of memory in float16

2. **Relatively slow**. As some or most of the operations are memory-bound (e.g., softmax), the large number of memory accesses translates to slow wall-clock time.

Meet the hero: Flash Attention!



Key ideas of Flash Attention

- **Tiling**
  Divide the attention weight matrix to tiles or blocks. Compute them separately.

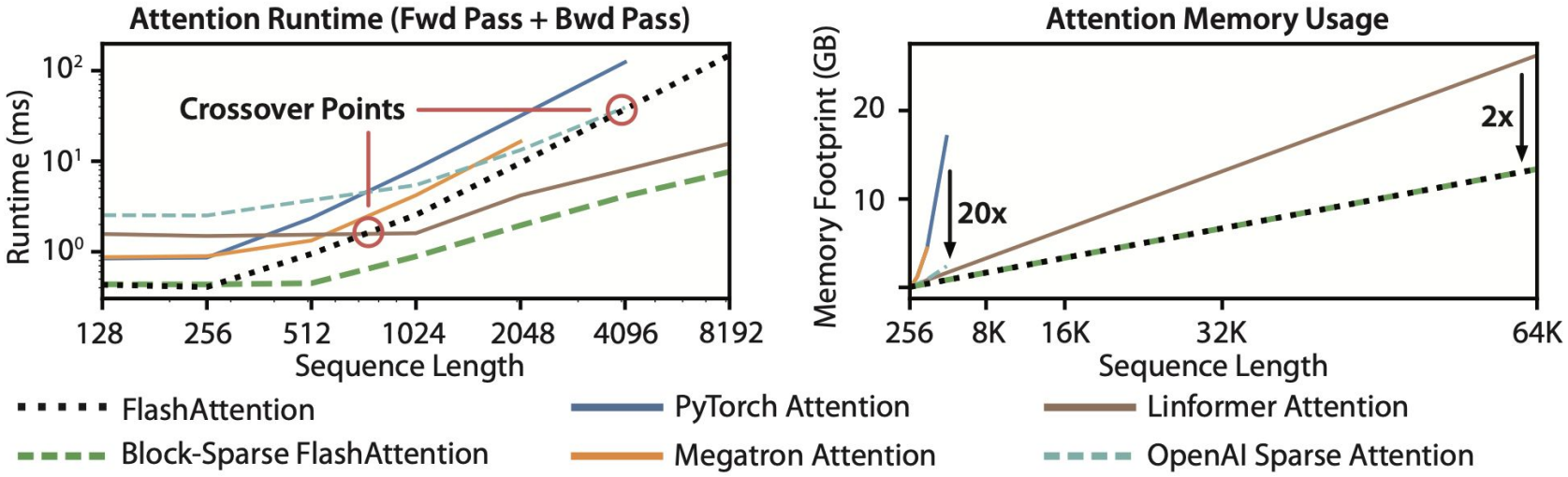
- **Recomputation**
  Do not save $S = QK^T$ (attention weight) matrix to memory for backward pass. Recompute it instead.

- **Fusing**
  Compute it as one CUDA kernel.

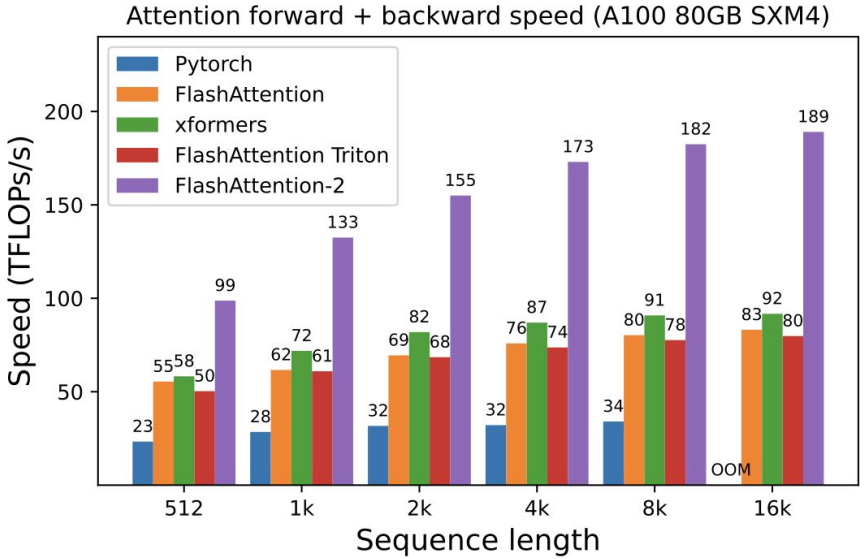
Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

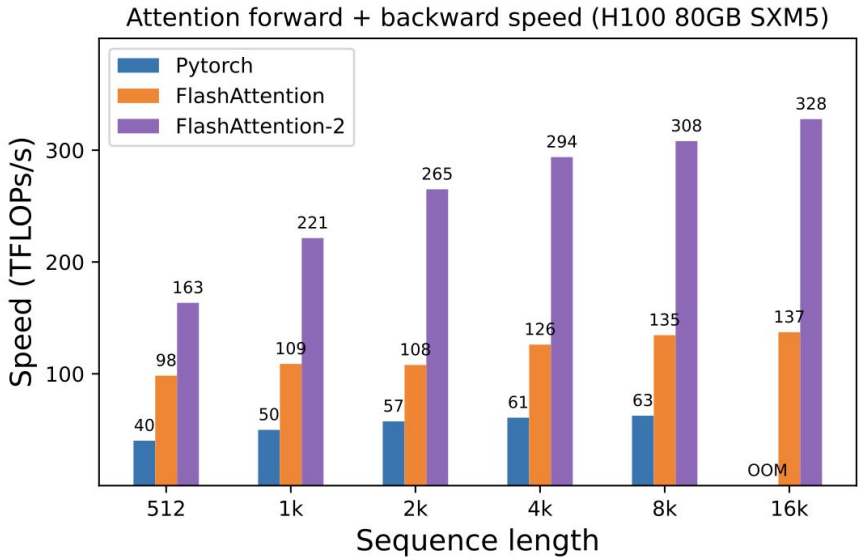
Results for seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU

Flash Attention 1



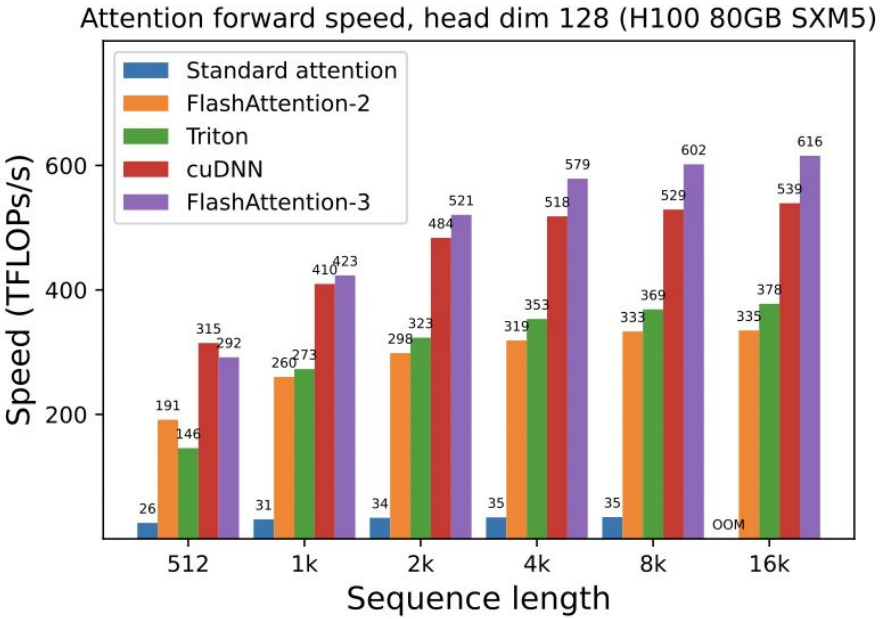
Flash Attention 2



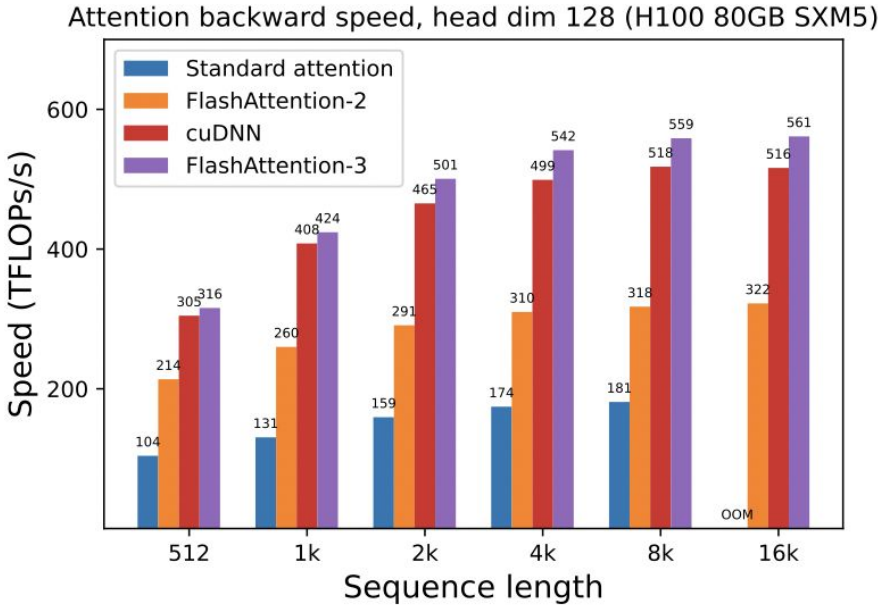


Credits: https://arxiv.org/abs/2307.08691

Flash attention 3 (H100 and H200 only)

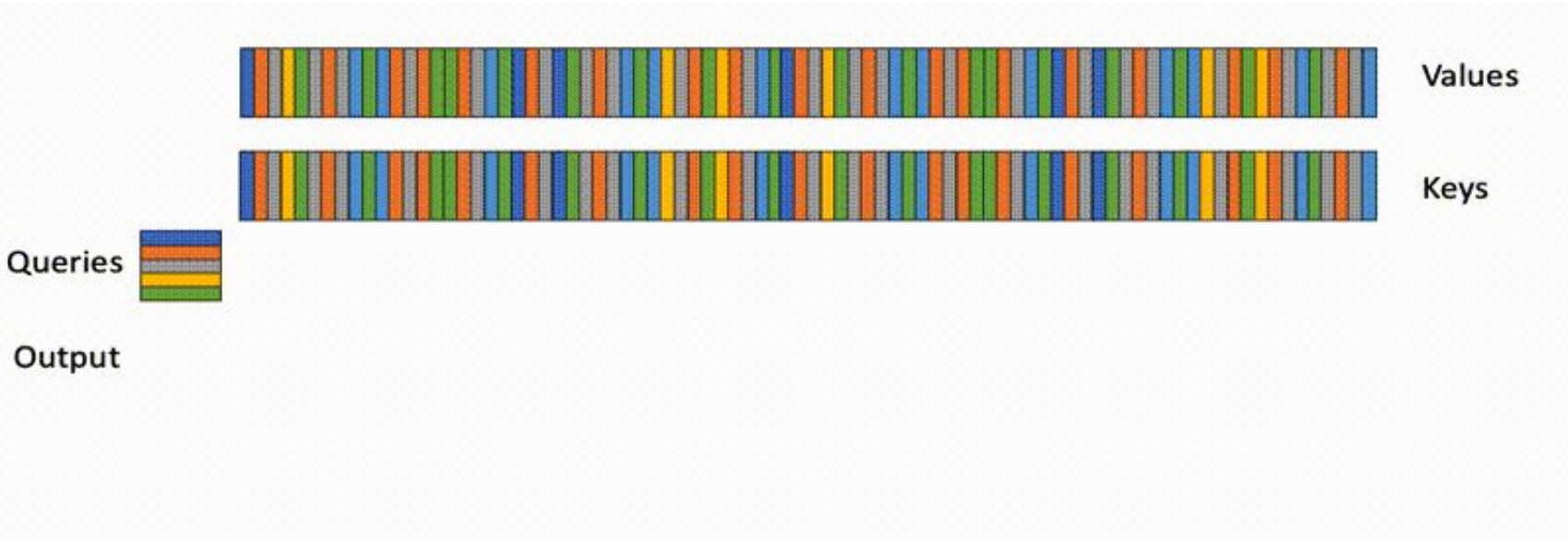


(d) Forward, with causal mask, head dim 128



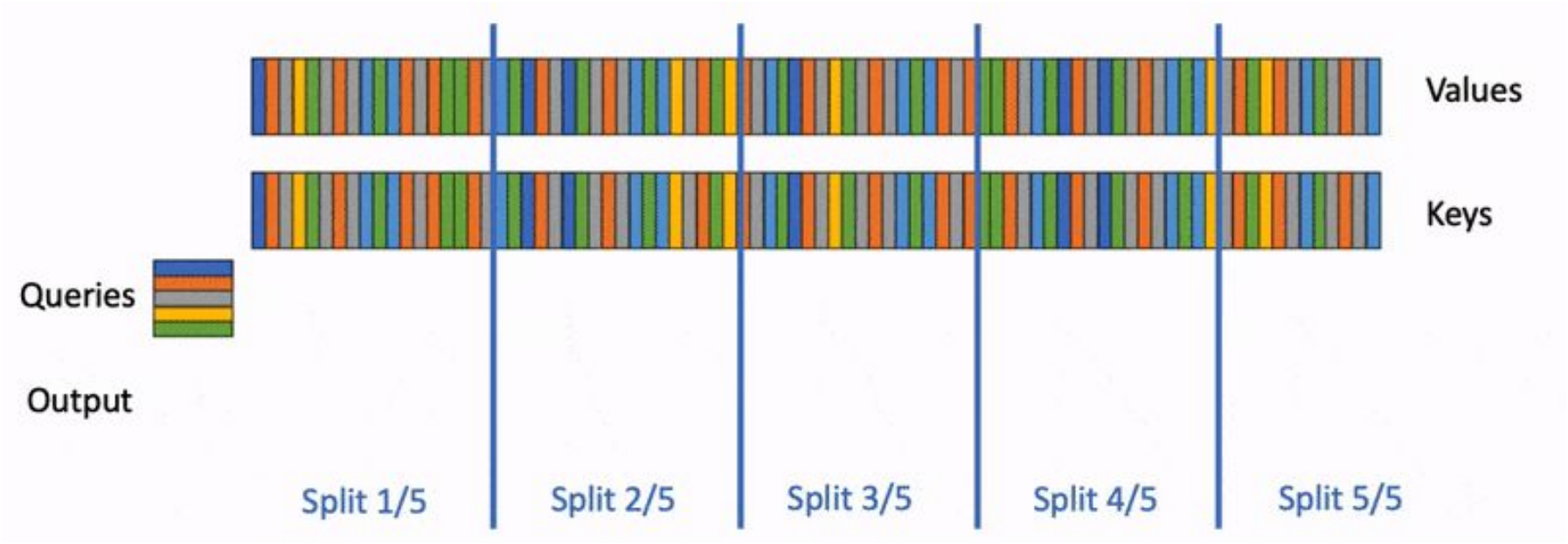
(b) Backward, without causal mask, head dim 128

Flash Attention during inference



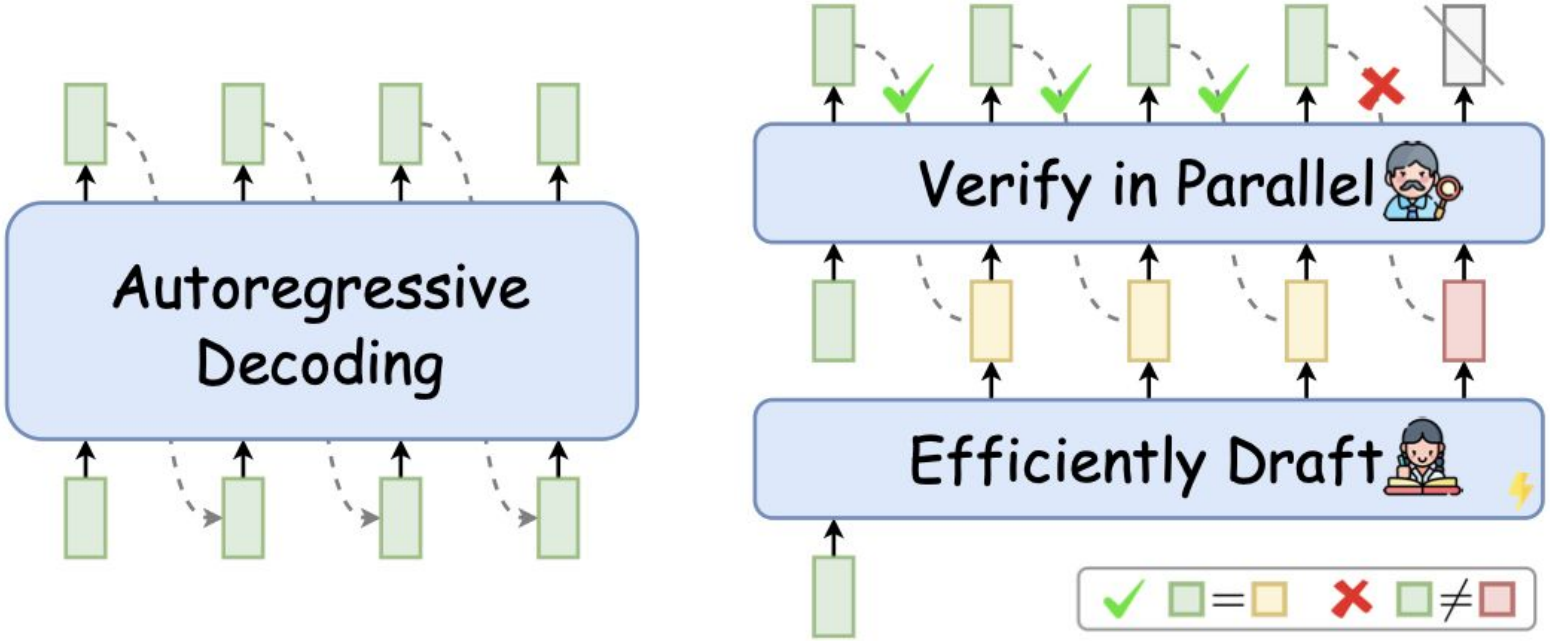
Credits: https://crfm.stanford.edu/2023/10/12/flashdecoding.html

Flash Attention for inference: Flash decoding



Credits: https://crfm.stanford.edu/2023/10/12/flashdecoding.html

Model optimization: Speculate decoding

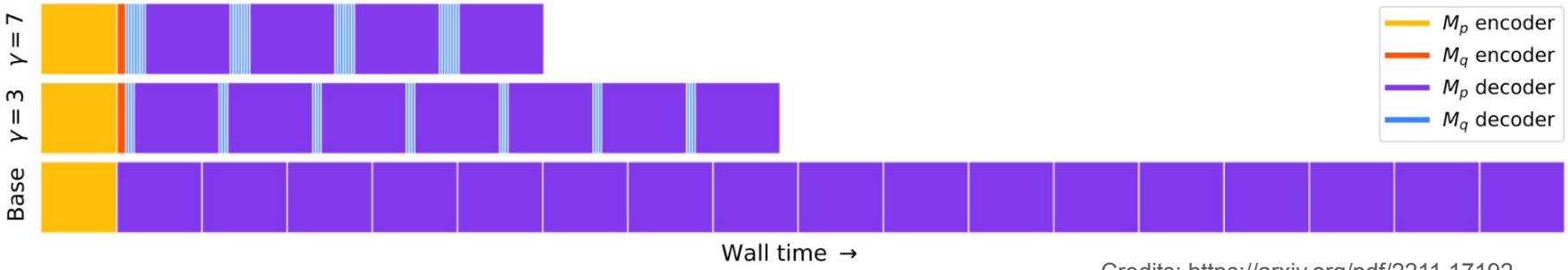


Credits: https://arxiv.org/pdf/2401.07851

Speculate decoding

There are two steps:

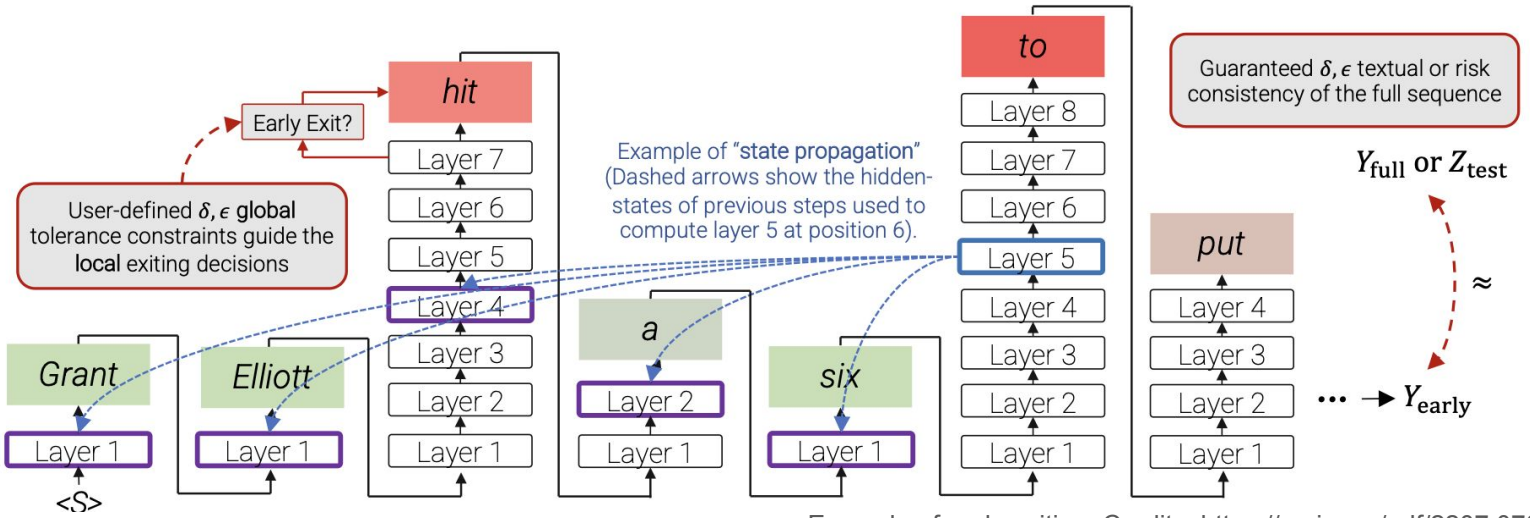
1. **Drafting**
   Generating new tokens to verify them in parallel with a target model
2. **Verification**
   Verifying drafted tokens in parallel to ensure the output quality is consistent with the target model



Credits: https://arxiv.org/pdf/2211.17192

Drafting strategies

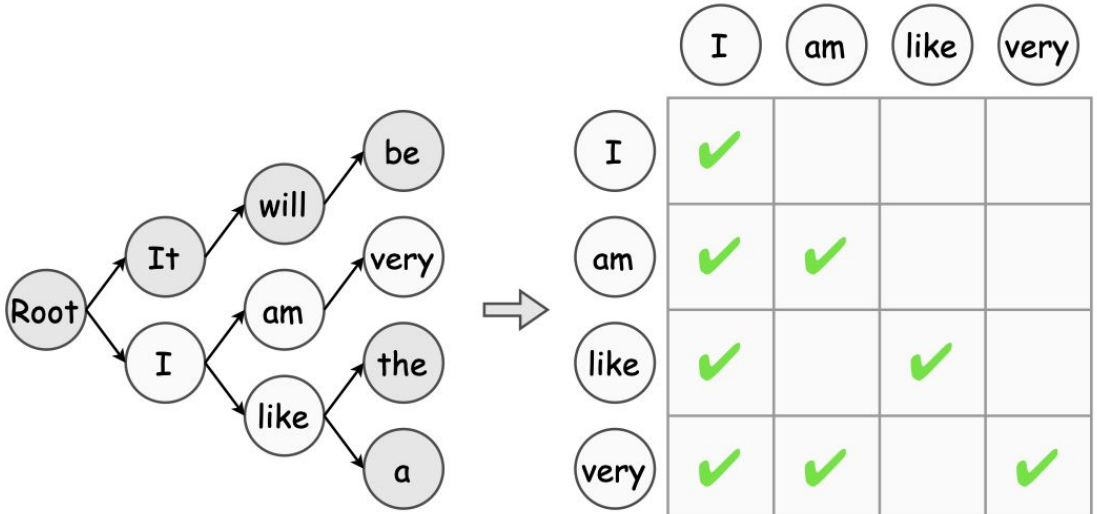
- Autoregressive drafting
  - Small LLM
  - Early exiting
  - Layer skipping
- Self-drafting
  - FFN heads
  - Non-autoregressive models
  - Mask predict



Example of early exiting. Credits: https://arxiv.org/pdf/2207.07061

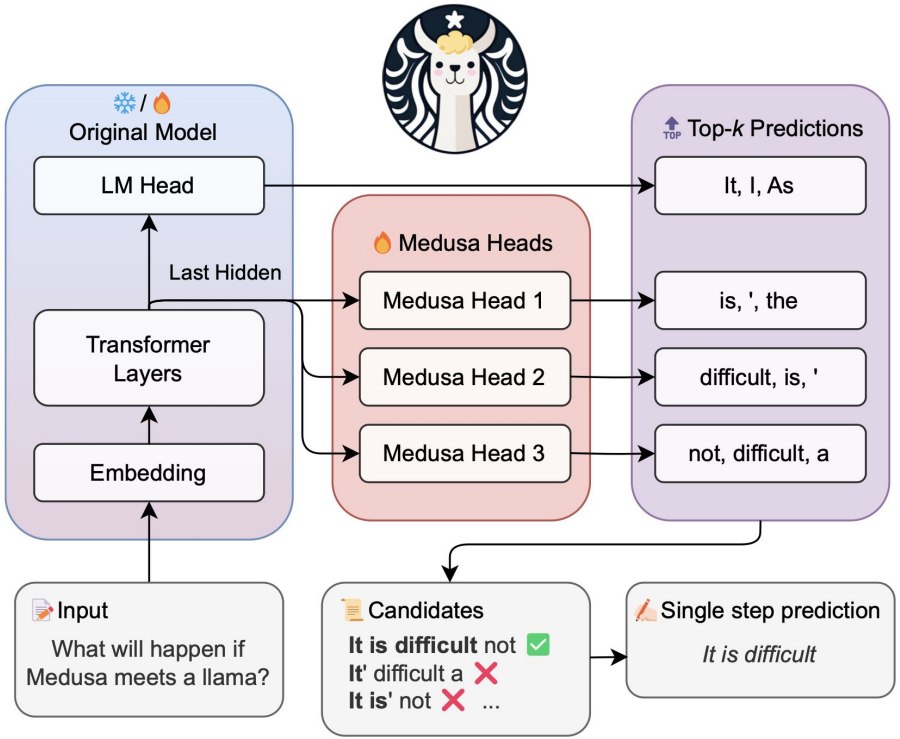
Verification strategies

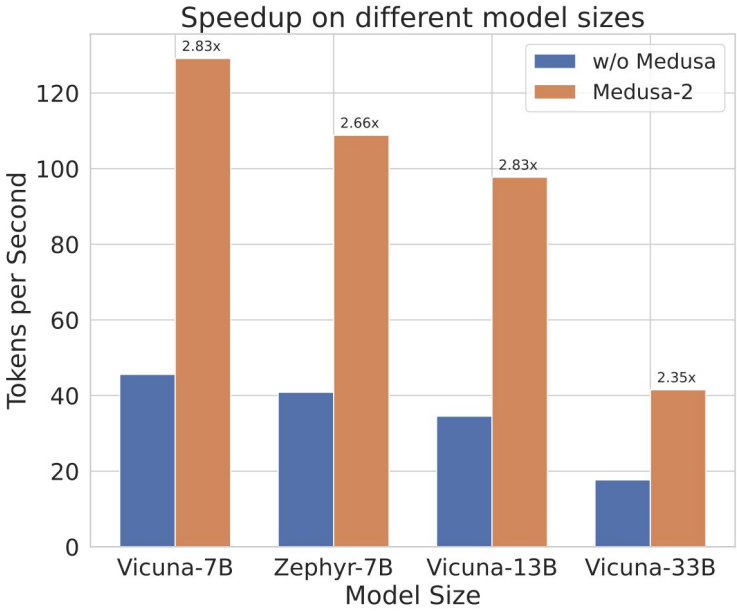
- Greedy decoding
- Nucleus sampling (top p)
- Token-tree verification



Example of token-free verification. Credits: https://arxiv.org/pdf/2401.07851

Speculative decoding example: Medusa

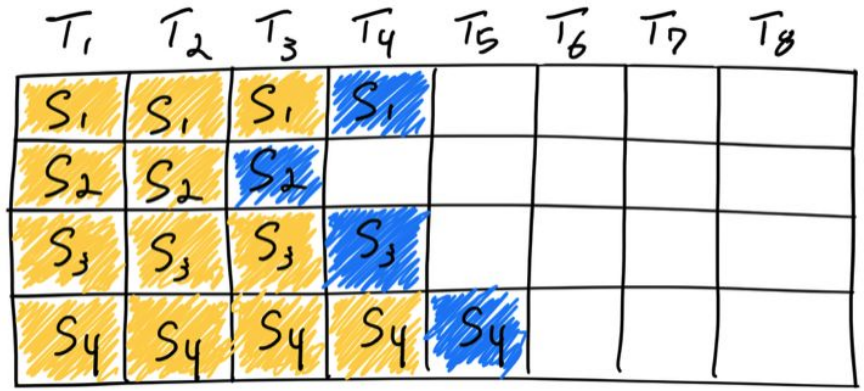


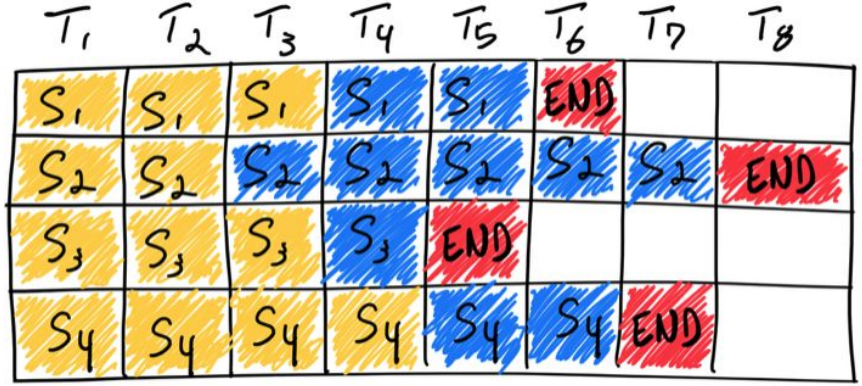


Credits: https://arxiv.org/abs/2401.10774

What’s wrong with batching?

Static batching problem: We need to wait until the most long sequence ended, which means that the GPU will be unutilized

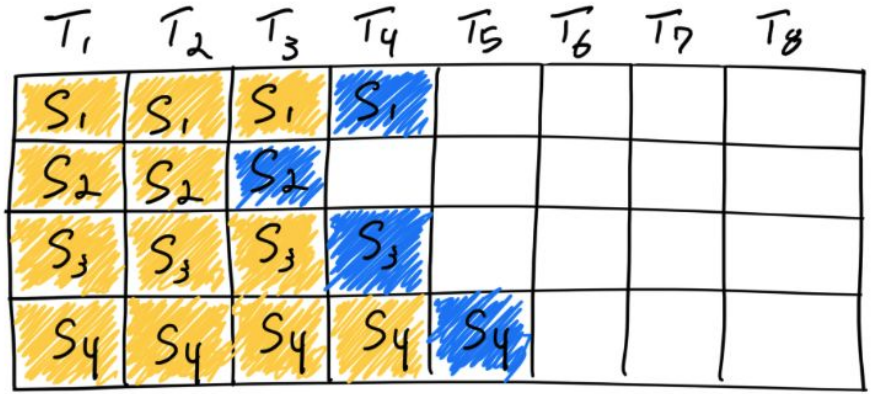


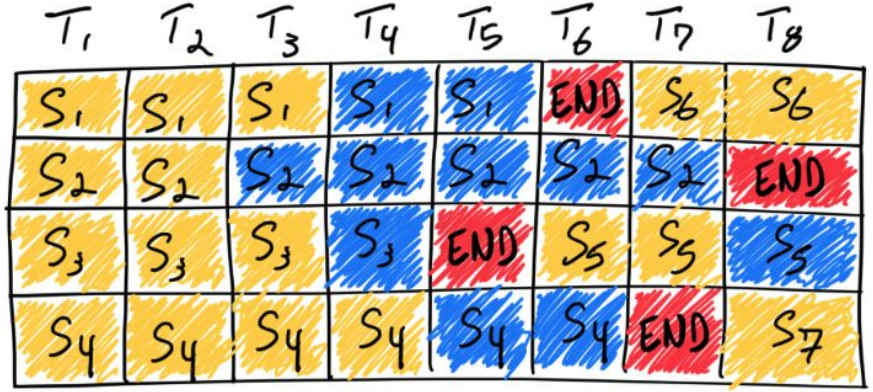


Credits: https://www.anyscale.com/blog/continuous-batching-llm-inference

Continuous batching

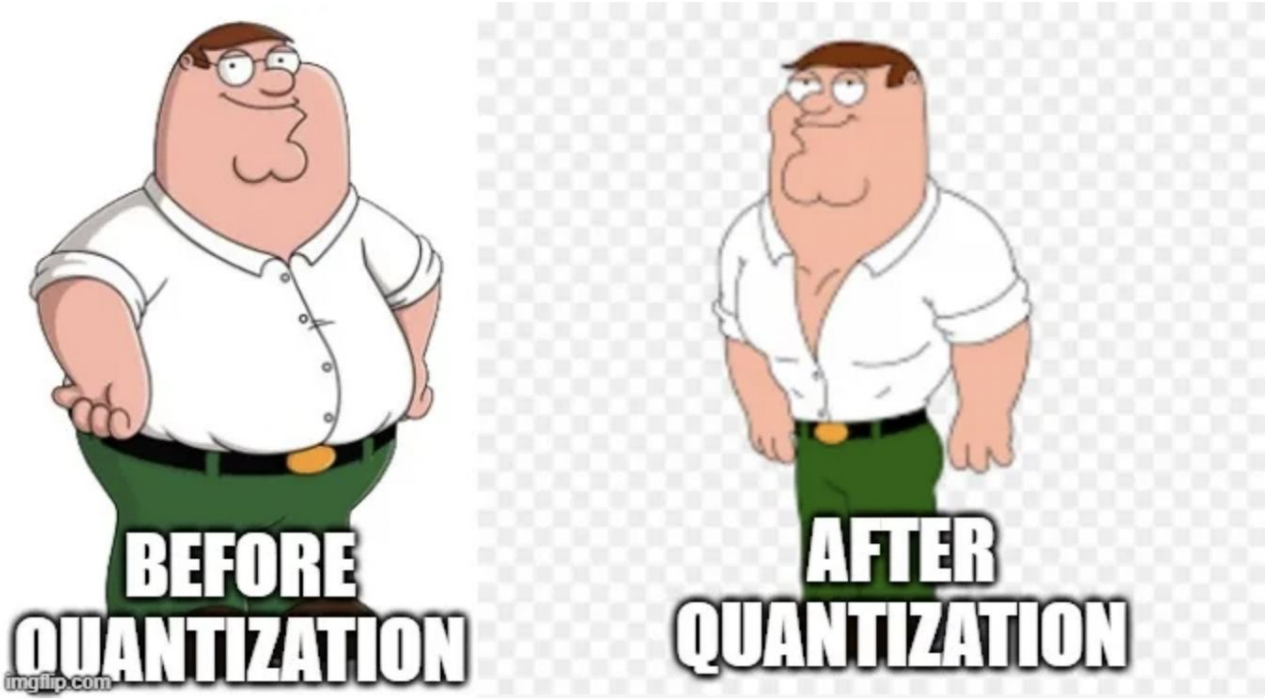
Wait for new requests with some time window, process (prefill) them simultaneously and replace already ended responses by the new ones





Credits: https://www.anyscale.com/blog/continuous-batching-llm-inference

Quantization



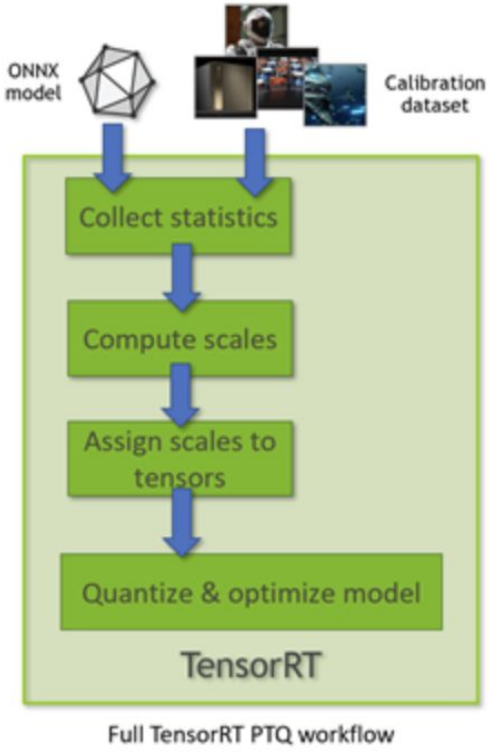
Credits: https://medium.com/kgxperience/model-quantization-who-wouldnt-want-their-models-slimmer-9f05b52da86b

Quantization of a model

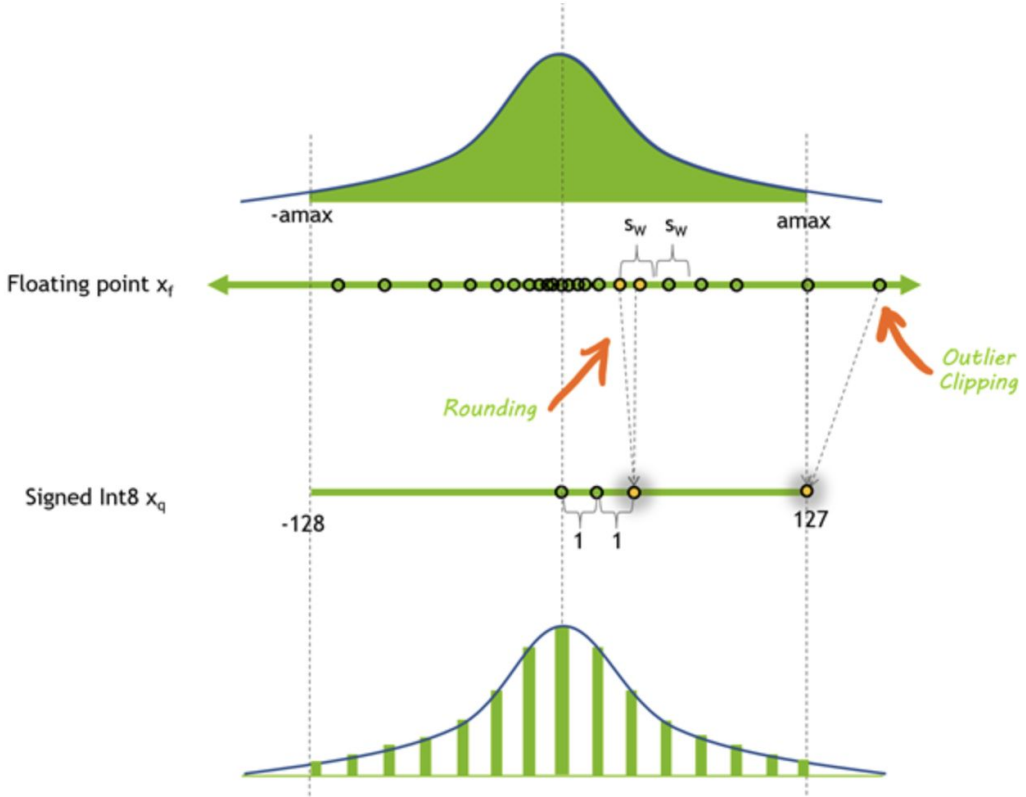
- Post training quantization

- Quantization aware training

Post training quantization

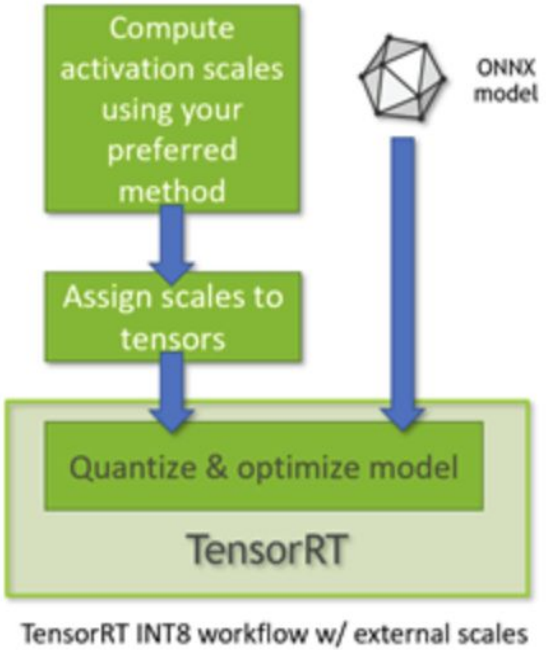


Full TensorRT PTQ workflow

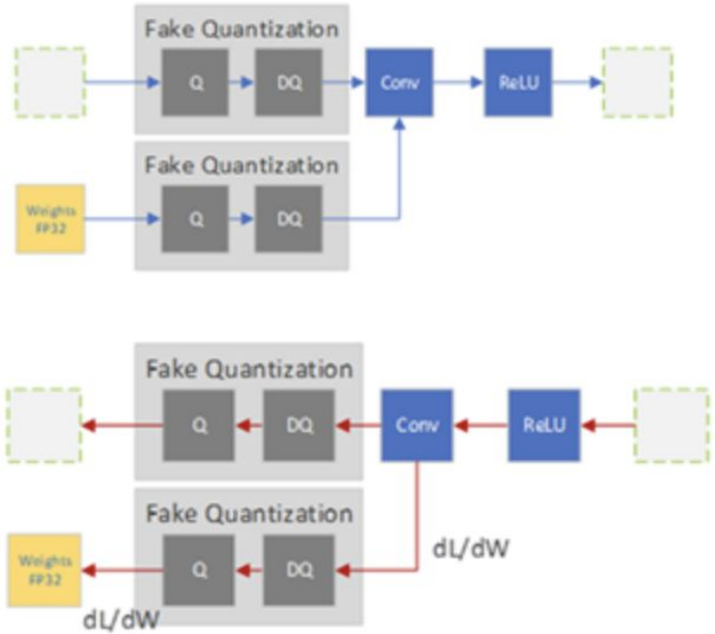


Credits: https://developer.nvidia.com/blog/achieving-fp32-accuracy-for-int8-inference-using-quantization-aware-training-with-tensorrt/

Quantization aware training



TensorRT INT8 workflow w/ external scales



Credits: https://developer.nvidia.com/blog/achieving-fp32-accuracy-for-int8-inference-using-quantization-aware-training-with-tensorrt/

Quantization of LLM

There are post-training activation methods that quantize only the weights, while leaving activations in float16/bfloat16.

This approach significantly reduces GPU memory consumption for large language model (LLM) weights. However, it may slightly increase inference time and lead to a marginal reduction in performance metrics on benchmarks.

Examples of such methods include:

- GPTQ
- Activation-aware Weight Quantization (AWQ)

Quantization of LLM: results

Model	batch=1
Yi-6B-Chat	12 GB
Yi-6B-Chat-4bits	4 GB
Yi-6B-Chat-8bits	7 GB
Yi-34B-Chat	65 GB
Yi-34B-Chat-4bits	19 GB
Yi-34B-Chat-8bits	35 GB

Credits: https://github.com/01-ai/Yi

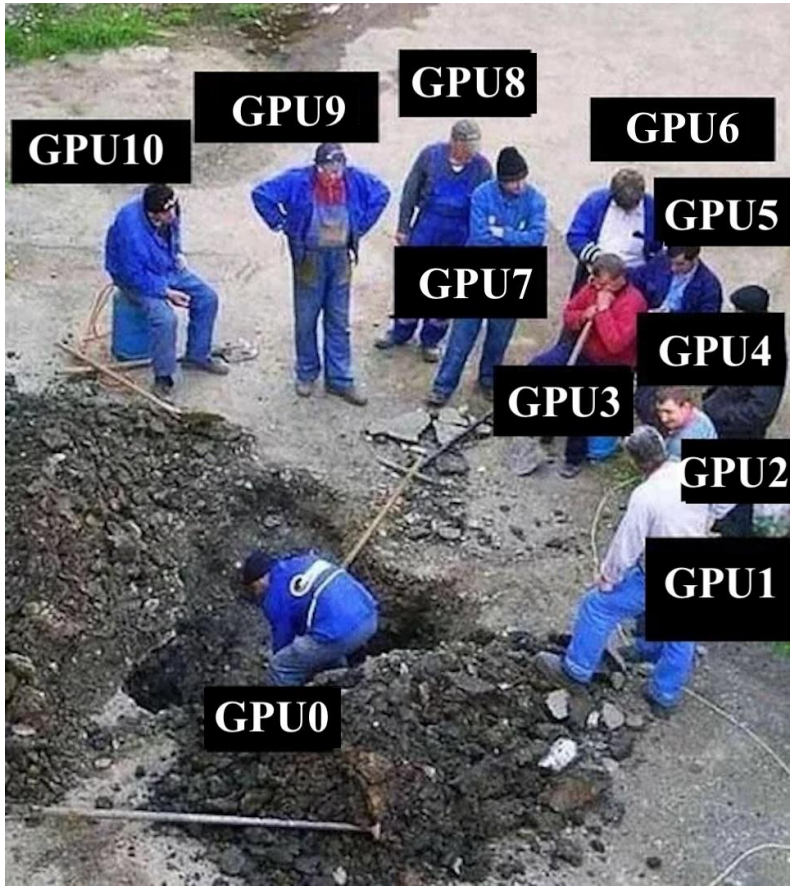
Model	MMLU
	0-shot
Yi-34B-Chat	**67.62**
Yi-34B-Chat-8bits (GPTQ)	66.24
Yi-34B-Chat-4bits (AWQ)	65.77
Yi-6B-Chat	58.24
Yi-6B-Chat-8bits (GPTQ)	58.29
Yi-6B-Chat-4bits (AWQ)	56.78

LLM Inference engines

- vLLM
- Nvidia Triton (TensorRT-LLM)
- Transformers

GPU Parallelism

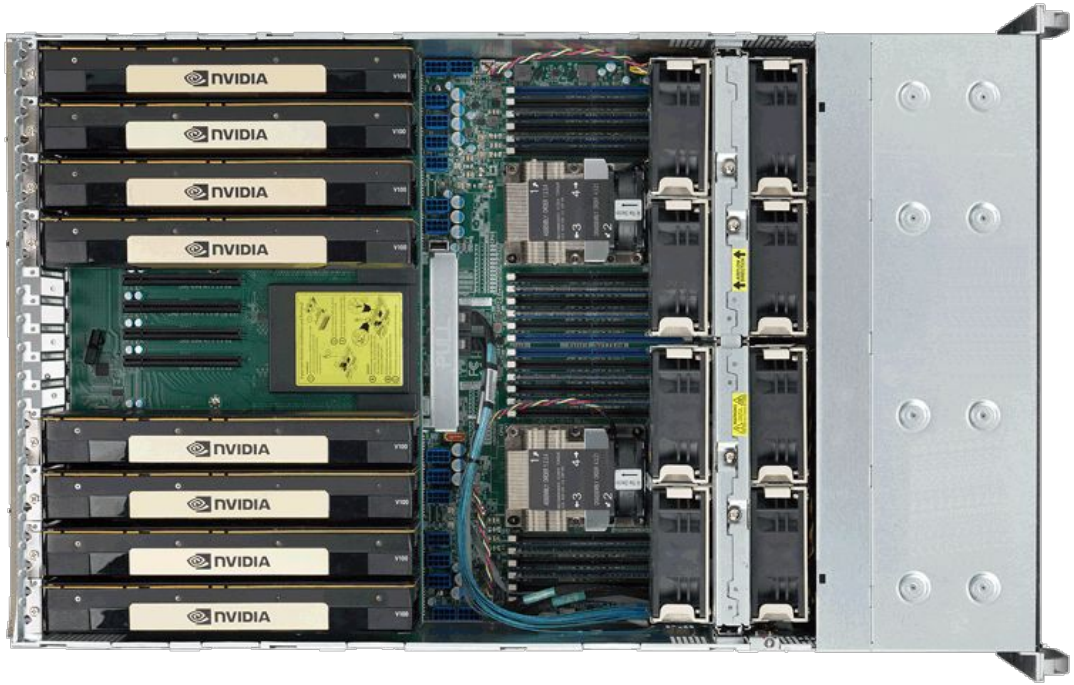
How to fit big LLM into multi GPU setup



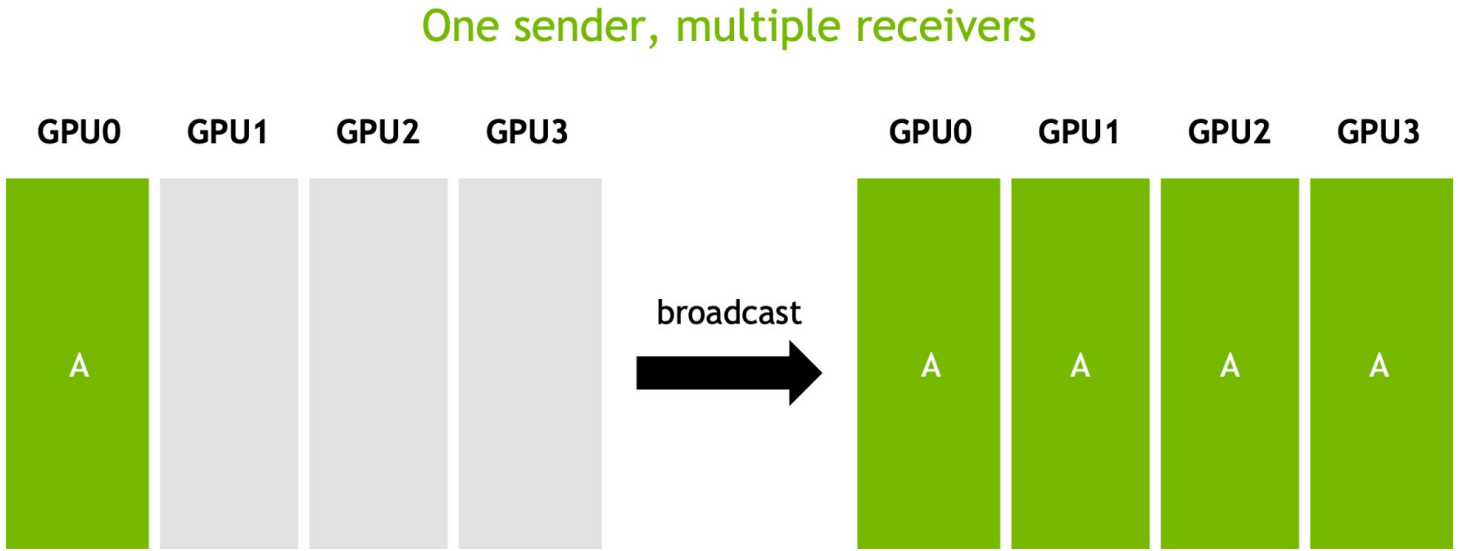
What is a GPU host?

Compute unit with 8 GPUs, connected via NvLink

GPU communication within a host is fast



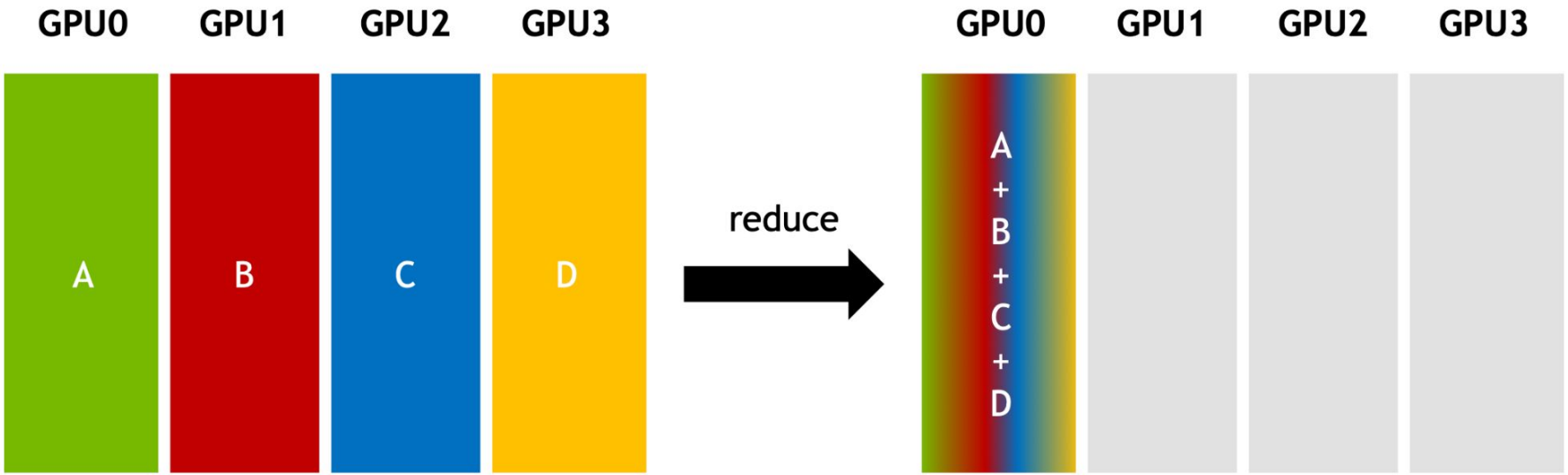
Collective communications (Broadcast)



Credits: https://images.nvidia.com/events/sc15/pdfs/NCCL-Woolley.pdf

Collective communications (Reduce)

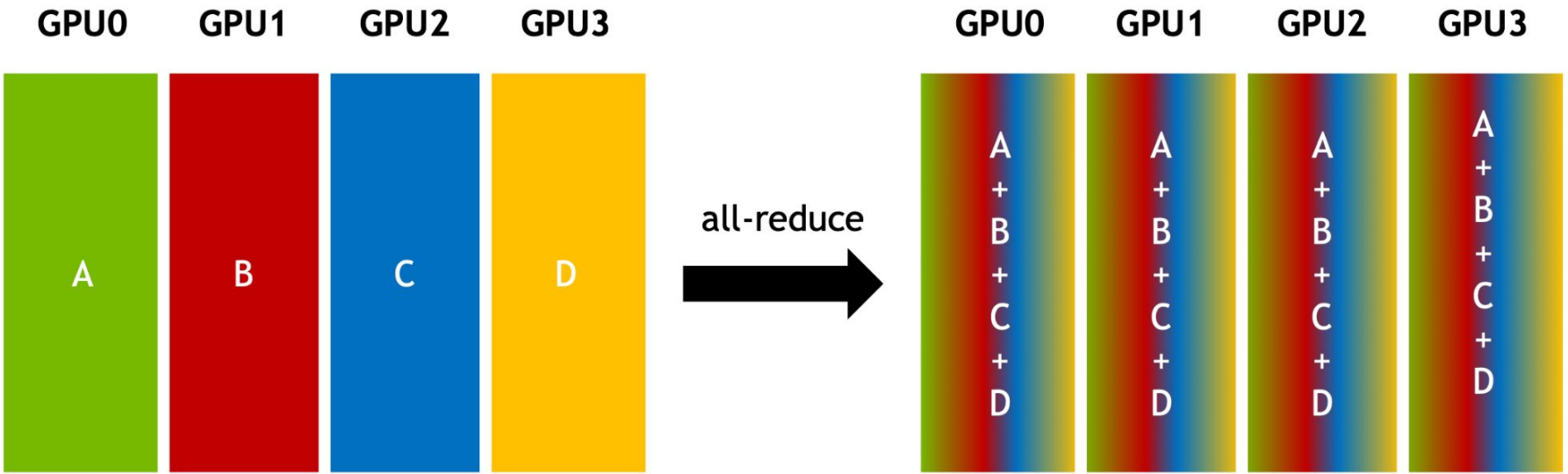
Combine data from all senders; deliver the result to one receiver



Credits: https://images.nvidia.com/events/sc15/pdfs/NCCL-Woolley.pdf

Collective communications (AllReduce)

Combine data from all senders; deliver the result to all participants

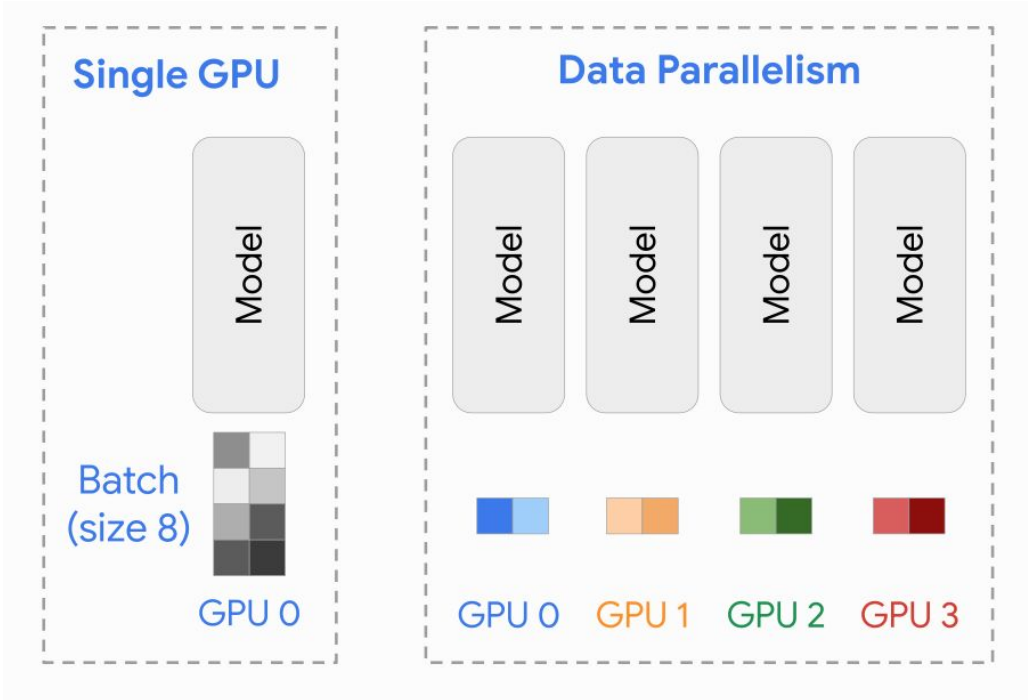


Credits: https://images.nvidia.com/events/sc15/pdfs/NCCL-Woolley.pdf

Data parallelism

- Using for relatively small LLMs such as <14B

- Very convenient when it's needed to infer a lot of data and you have a few free GPUs

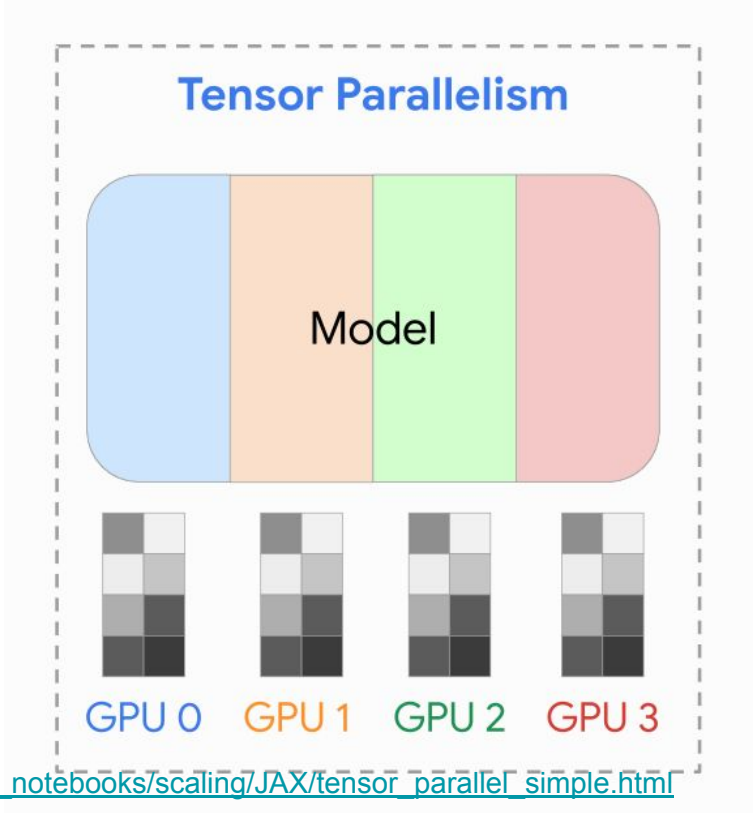


Credits: https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Tensor parallelism (Model parallelism)

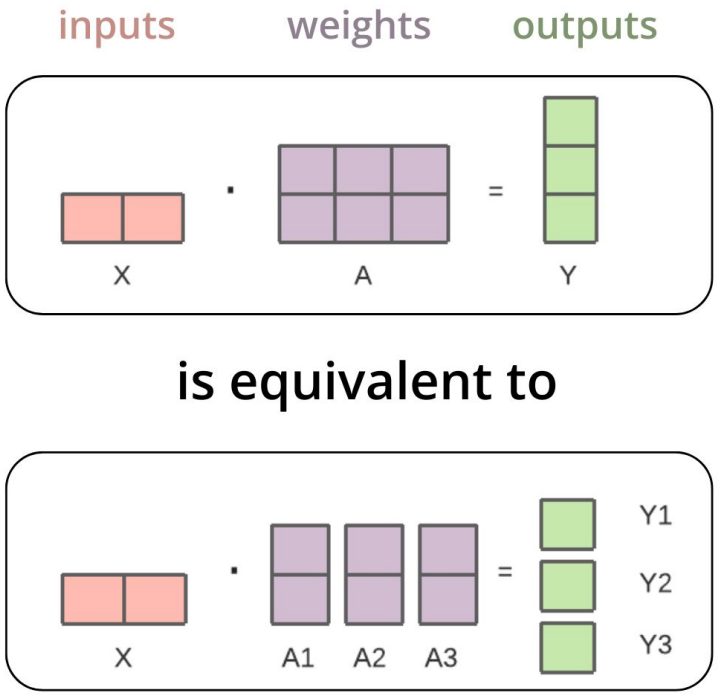
- Using for relatively big LLMs

- Best practice: to fit the model to one host (8 GPUs)



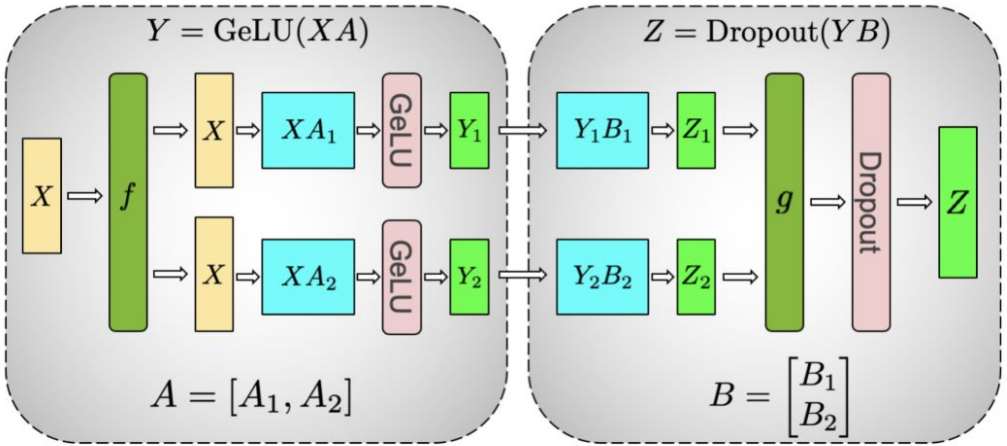
Credits: https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Tensor parallelism in depth

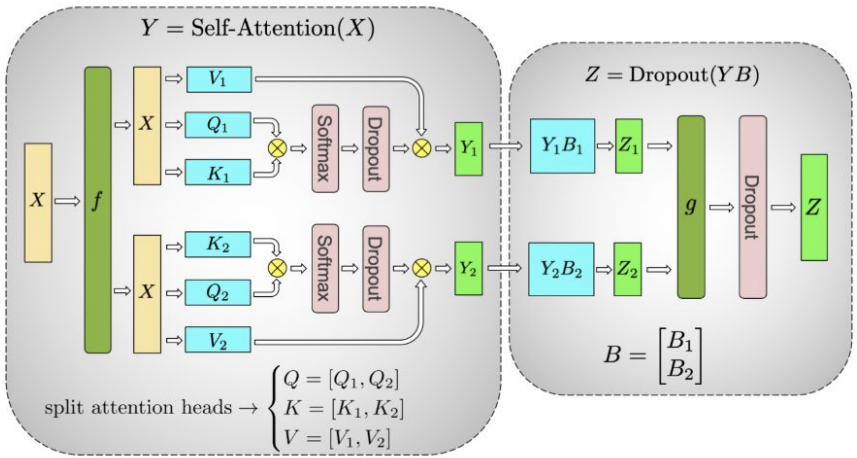


Credits: https://huggingface.co/docs/text-generation-inference/conceptual/tensor_parallelism

Tensor parallelism in depths



(a) MLP



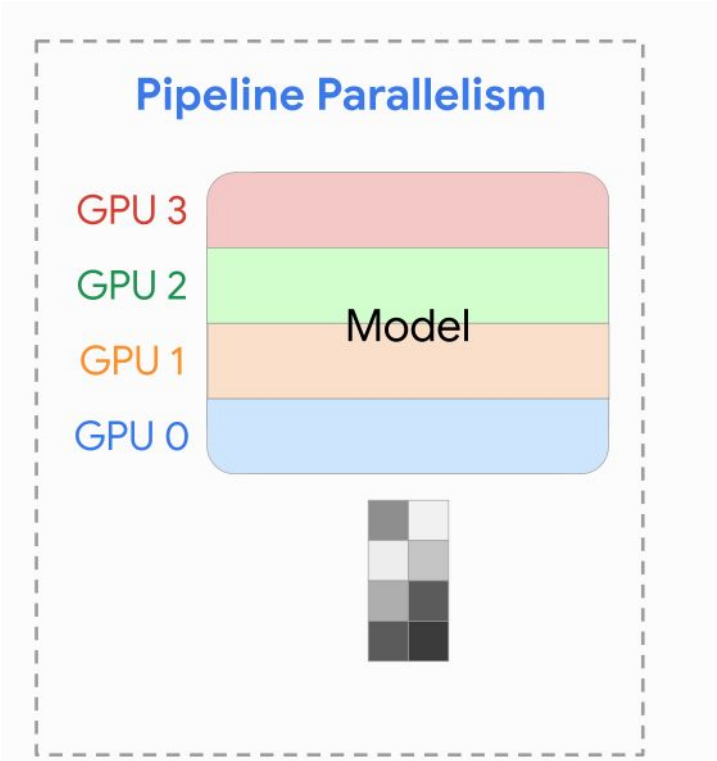
(b) Self-Attention

f = Broadcast
g = Reduce (AllReduce)

f = Broadcast
g = Reduce (AllReduce)

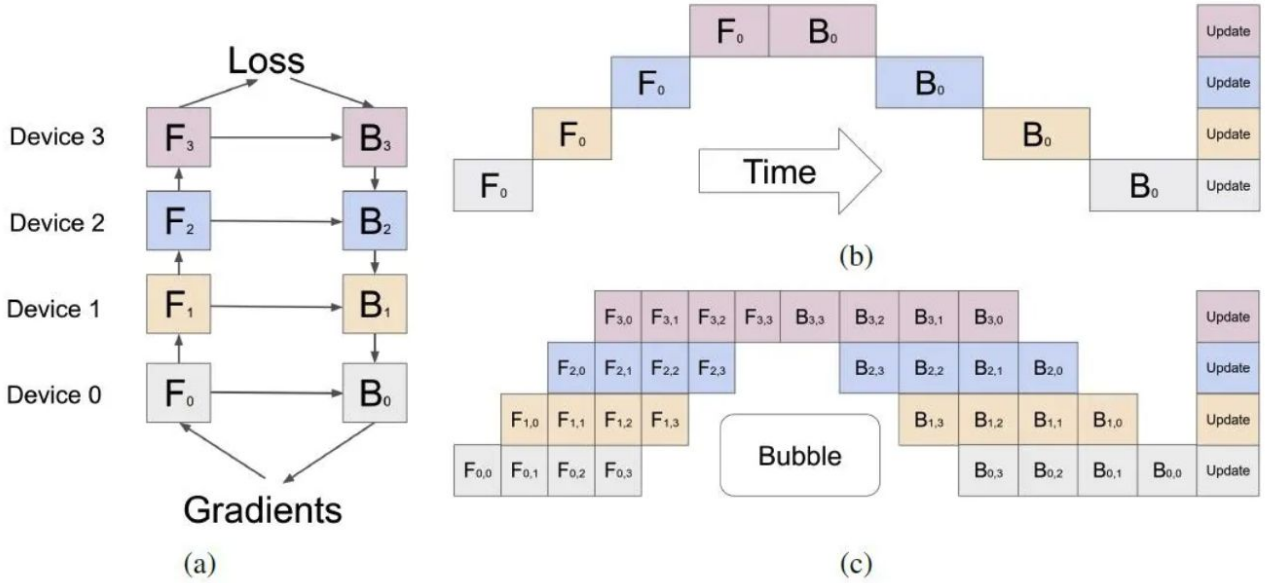
Pipeline parallelism

- Using for large LLMs which do not fit into a single host



Credits: https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Pipeline parallelism in depths



Credits: https://medium.com/nerd-for-tech/an-overview-of-pipeline-parallelism-and-its-research-progress-7934e5e6d5b8

Our course finished!

Thank you for your attention!